Software Reliability

Chapter 7 — Advanced Software Engineering

University: University of Wasit
College: College of Computer Science and Information Technology
Prepared by: Abul-Hassan Abbas Khudair
Supervised by: Asst. Prof. Dr. Ali Fahim Ni'ma
Year: 2026–2027

Table of Contents

7. Software Reliability 4
7.1 Basic Concepts 4
7.1.1 What is Software Reliability? 5
7.1.2 Software Reliability and Hardware Reliability 6
7.1.3 Failures and Faults 6
7.1.4 Environment 9
7.1.5 Uses of Reliability Studies 12
7.1.6 Reliability at the System Level 12
7.1.7 Reliability Terminologies 13
7.1.8 Classification of Failures 13
7.1.9 Reliability Metrics 14
7.1.10 Why Software Reliability is Harder to Measure than Hardware Reliability 15
7.2 Software Quality 16
7.2.1 McCall Software Quality Model 19
7.2.2 Boehm Software Quality Model 22
7.2.3 ISO 9126 23
7.2.4 The Quality Factors in Detail 26
7.2.5 Measuring the Quality Factors 27
7.2.6 Software Quality Assurance 28
7.2.7 Quality Metrics in Practice 29
7.2.8 Beyond ISO 9126: the ISO 25010 View 29
7.2.9 The FURPS Quality Model 30
7.2.10 Representative Qualities 31
7.2.11 The SQA Plan 33
7.2.12 Classification of Software Qualities 34
7.2.13 Why Software Quality Matters 34
7.2.14 Fitness of Purpose and the Modern View of Quality 35
7.2.15 From Quality Factors to Quantitative Measures 36
7.2.16 Three Views of Quality 36
7.3 Software Reliability Models 37
7.3.1 Basic Execution Time Model 38
7.3.2 Logarithmic Poisson Execution Time Model 42

7.3.3 Calendar Time Component 45
7.3.4 The Jelinski-Moranda Model 49
7.3.5 The Bug Seeding Model 50
7.3.6 Choosing Among the Models 52
7.3.7 Reliability Growth Models 52
7.3.8 Reliability Assessment 53
7.3.9 Statistical Testing and the Operational Profile 54
7.3.10 Uses of Reliability Studies 55
7.3.11 Verification, Validation and Reliability 56
7.4 Capability Maturity Model 56
7.4.1 Maturity Levels 57
7.4.2 Key Process Areas 59
7.4.3 Common Features 60
7.4.4 Process Capability and Process Performance 61
7.4.5 Benefits and Limitations 62
7.5 ISO 9000 62
7.5.1 Mapping ISO 9001 to the CMM 63
7.5.2 Contrasting ISO 9001 and the CMM 64
7.5.3 Conclusion 65
7.5.4 Obtaining ISO 9000 Certification 65
7.5.5 Benefits and Limitations of ISO 9000 Certification 66
7.5.6 Salient Features of ISO 9001 Requirements 67
7.5.7 The ISO 9000 Family and Its Mission 68
7.5.8 ISO 9000 Certification and the SEI-CMM Compared 69
7.5.9 Why the Software Industry Seeks Certification 70

7. Software Reliability

Software reliability. Does it exist? Computer program instructions cannot break or wear out. Hence predicting software reliability is a different concept as compared to predicting hardware reliability. Software becomes more reliable over time, instead of wearing out. It becomes obsolete as the environment for which it was developed changes. Hardware redundancy allows us to make a system reliable as we desire, if we use large number of components with given reliability. We do not have such techniques in software and we may not get such techniques in foreseeable future.

Initially, problems in programming were blamed to the severe constraints imposed by the hardware. However, hardware has now become more reliable, flexible and versatile, but the problems with programming have not decreased.

7.1 Basic Concepts

The term reliability is often misunderstood in the software field since software does not break or wear-out in the physical sense. It either works in a given environment or it does not. Hence traditional "bath tub" curve of hardware reliability is not applicable here. The "bath tub curve" is given in Fig. 7.1.

As indicated, there are three phases in the life of any hardware component i.e., burn-in, useful life and wear-out. In burn-in phase, failure rate is quite high initially, and it starts decreasing gradually as the time progresses. It may be due to initial testing in the premises of the organisation. During useful life period, failure rate is approximately constant. Failure rate increases in wear-out phase due to wearing out/aging of components. The best period is useful life period. The shape of this curve is like a "bath tub" and that is why it is known as bath tub curve.

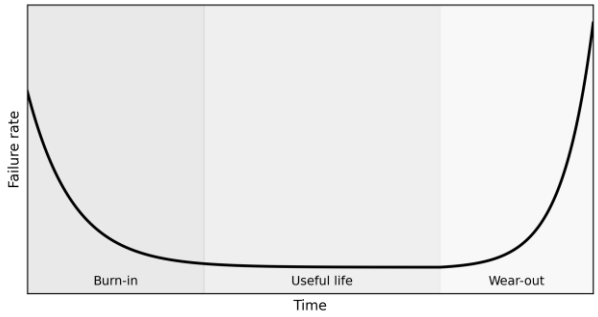


Figure 7.1. Bath tub curve of hardware reliability.

We do not have wear out phase in software. The expected curve for software is given in Fig. 7.2.

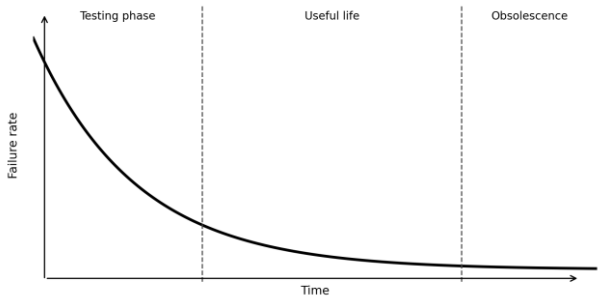


Figure 7.2. Software reliability curve (failure rate versus time).

Software may be retired only if it becomes obsolete. Some of contributing factors are given below:

- change in environment
- change in infrastructure/technology
- major change in requirements
- increase in complexity
- extremely difficult to maintain
- deterioration in structure of the code
- slow execution speed
- poor graphical user interfaces

7.1.1 What is Software Reliability?

According to Bev Littlewood [LITT 79]: "Software reliability means operational reliability. Who cares how many bugs are in the program? We should be concerned with their effect on its operations".

As per IEEE standard [IEEE 90]: "Software reliability is defined as the ability of a system or component to perform its required functions under stated conditions for a specified period of time".

Software reliability is also defined as the probability that a software system fulfills its assigned task in a given environment for a predefined number of input cases, assuming that the hardware and the inputs are free of error [KOPE 79]. Hence it is the probability that the software will work without failure for a specified period of time in a given environment. Here environment and time is fixed. Reliability is for fixed time under given environment or stated conditions. So, reliability value is always for well defined domain.

The most acceptable definition of software reliability is: "It is the probability of a failure free operation of a program for a specified time in a specified environment [MUSA87]".

For example, a time-sharing system may have a reliability of 0.95 for 10 hr when employed by the average user. This system, when executed for 10 hr, would operate without failure for 95 of these periods out of 100. As a result of the general way in which we defined failure, note that the concept of software reliability incorporates the notion of performance being satisfactory. For example, excessive response time at a given load may be considered unsatisfactory, so that a routine must be recorded in more efficient form.

7.1.2 Software Reliability and Hardware Reliability

The field of hardware reliability has been established for some time. Hence, one might ask how software reliability relates to it. In reality, the division between hardware and software reliability is somewhat artificial. Both may be defined in the same way. Therefore, one may combine hardware and software component reliabilities to get system reliability. Both depend on the environment. The source of failures in software is design faults, while the principal source in hardware has generally been physical deterioration. However, the concepts and theories developed for software reliability could really be applied to any design activity, including hardware design.

Once a software (design) defect is properly fixed, it is in general fixed for all time. Failure usually occurs only when a program (design) is exposed to an environment that it was not developed or tested for. Although manufacturing can affect the quality of physical components, the repetition process for software (design) is trivial and can be performed to very high standards of quality. Since introduction and removal of design faults occurs during software development, software reliability may be expected to vary during this period.

The design reliability concept has not been applied to hardware to that extent. The probability of failure due to wear and other physical causes has usually been much greater than that due to an unrecognised design problem. It was possible to keep hardware design faults low because hardware was generally less complex logically than software. Hardware design faults had to be kept low because retrofitting of manufactured items in the field was very expensive. Awareness of the work that is going on in software reliability, plus a growing realisation of the importance of design faults, may now be having an effect on hardware reliability too. This growing awareness is strengthened by the parallels that people are starting to draw between software engineering and chip design.

A final characteristic of software reliability is that it tends to change continually during test periods. This happens either as new problems are introduced, when new code is written or when repair actions removes problems that exist in the code. Hardware reliability may change during certain periods, such as initial burn-in or the end of useful life. However, it has a much greater tendency to snowball toward a constant value.

Despite the foregoing differences, we can develop software reliability theory in a way that is compatible with hardware reliability theory. Thus system reliability figures may be computed using standard hardware combinatorial techniques [SHOO86]. Hardware and software reliability share many similarities and some differences [LLO77]. One must not err on the side of assuming that software always presents unique problems, but one must also be careful not to carry analogies too far.

7.1.3 Failures and Faults

What do we mean by the term software failure? It is the departure of the external results of the program operation from requirements. So our failure is something dynamic. The program has to be executing for a failure to occur. The term failure relates to the behaviour of the program. This performance degradation of failure is deliberate. It can include such things as deficiency in performance attributes and excessive response time.

A fault is the defect in the program that, when executed under particular conditions, causes a failure. There can be different set of conditions that cause failures, or the conditions can be repeated. Hence a

fault can be the source of more than one failure. A fault is a property of the program rather than a property of its execution or behaviour. It is what we are referring to in general when we use the term bug. A fault is created when a programmer makes an error. It's very important to make the failure-fault distinction.

Reliability quantities have usually been defined with respect to time, although it would be possible to define them with respect to other variables. We are concerned with three kinds of time. The execution time for a program is the time that is actually spent by a processor in executing the instructions of that program. The second kind of time is calendar time. It is the familiar time that we normally experience. Execution time is important, because it is now well accepted that models based on execution time are superior. However, quantities generally have been related back to calendar time to be meaningful to engineers or managers. Sometimes the term clock time is used for a program. It represents the elapsed time from start to end of program execution on a running computer. It includes wait time and the execution of other programs. Periods during which the computer is shut down are not counted. If computer utilisation by the program, which is the fraction of time the processor is executing program, is constant, clock time will be proportional to execution time.

There are four general ways of characterising failure occurrences in time:

1. time of failure,
2. time interval between failures,
3. cumulative failures experienced up to a given time,
4. failures experienced in a time interval.

These are illustrated in Tables 7.1 and 7.2.

Table 7.1: Time based failure specification

Failure Number	Failure Time (sec)	Failure Interval (sec)
1	8	8
2	18	10
3	25	7
4	36	11
5	45	9
6	57	12
7	71	14
8	86	15
9	104	18
10	124	20
11	143	19
12	169	26
13	197	28
14	222	25
15	250	28

Table 7.2: Failure based failure specification

Time (sec)	Cumulative Failures	Failure in interval (30 sec)
30	3	3
60	6	3
90	8	2
120	9	1

150	11	2
180	12	1
210	13	1
240	14	1

Table 7.3: Probability distribution at times tA and tB

Value (failures in time period)	Probability at tA = 1 hr	Probability at tB = 5 hr
0	0.10	0.01
1	0.18	0.02
2	0.22	0.03
3	0.16	0.04
4	0.11	0.05
5	0.08	0.07
6	0.05	0.09
7	0.04	0.12
8	0.03	0.16
9	0.02	0.13
10	0.01	0.10
11	0	0.07
12	0	0.05
13	0	0.03
14	0	0.02
15	0	0.01
Mean failures	3.04	7.77

Table 7.3 illustrates a typical probability distribution of failures that occurs within a time period of execution. Each possible value of the random variable of number of failures is given alongwith its associated probability. The probabilities, of course, add to 1. Note that here the random variable is discrete, as the number of failures must be an integer. Note that the most probable number of failures is 2 for t = 1 hr. The mean or average number of failures can be computed. We multiply each possible value by the probability it can occur and add all the products. The mean is 3.04 failures for t = 1 hour.

We will look at the time variation from two different viewpoints, the mean value function and the failure intensity function. The mean value function represents the average cumulative failures associated with each time point. The failure intensity function is the rate of change of the mean value function or the number of failures per unit time. For example, you might say 0.01 failure/hr or 1 failure/100 hr. Strictly speaking, the failure intensity is the derivative of the mean value function with respect to time, and is an instantaneous value.

A random process whose probability distribution varies with time is called non-homogeneous. Most failure processes during test fit this situation. Fig. 7.3 illustrates the mean value and the related failure intensity functions at time tA and tB. Note that the mean failures experienced increases from 3.04 to 7.77 between these two points, while the failure intensity decreases.

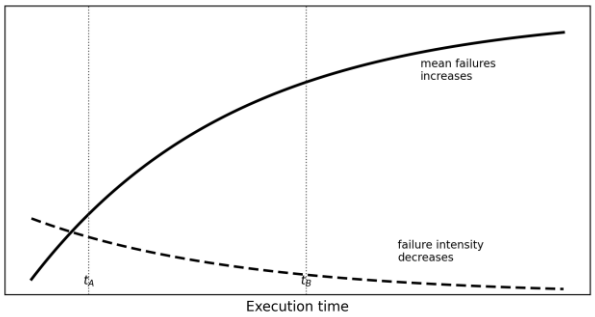


Figure 7.3. Mean value and failure intensity functions.

Failure behaviour is affected by two principal factors:

1. the number of faults in the software being executed,
2. the execution environment or the operational profile of execution.

The number of faults in software is the difference between the number of faults introduced and the number removed.

Faults are introduced when the code is being developed by programmers. They may introduce the faults during original design or when they are adding new features, making design changes, or repairing faults that have been identified. In general, only code that is modified or modified results in faults introduction. Code that is inherited from another application does not usually introduce any appreciable number of faults, except possibly in the interfaces. Note that the process of faults removal introduces some new faults because it involves modification or writing of new code.

Faults removal obviously can't occur unless you have some means of detecting the fault in the first place. Thus fault removal resulting from execution depends on the occurrence of the associated failure. Occurrence depends both on the length of time for which the software has been executing and on the execution environment or operational profile. When different functions are executed, different faults are encountered and the failures that are exhibited tend to be different; thus the environmental influence. We can often find faults without execution. They may be found through inspection, compiler diagnostics, design or code reviews, or code reading.

7.1.4 Environment

Let us scrutinise the term environment. The environment is described by the operational profile. We need to build up to the concept of the operational profile through several steps. It is possible to view the execution of a program as a single entity. The execution can last for months or even years for a real time system. However, it is more convenient to divide the execution into runs. The definition of run is somewhat arbitrary, but it is generally associated with some function that the program performs. Thus, it can conveniently describe the functional environment of the program. Runs that are identical repetitions of each other are said to form a run type. The proportion of runs of various types may vary, depending on the functional environment. Examples of a run type might be:

1. a particular transaction in an airline reservation system or a business data processing system,
2. a specific cycle in a closed loop control system (for example, in a chemical process industry),
3. a particular service performed by an operating system for a user.

During test, the term test case is sometimes used instead of run type.

We next need to understand the concept of the input variable. This is a variable that exists external to the program and is used by the program in executing its function. For an airline reservation, destination might be an input variable. One generally has a large quantity of input variables associated with the program, and each set of values of these variables characterise an input state.

In effect, the input state identifies the particular run type that you're making. Therefore, runs can always be classified by their input states. Again, taking the case of the airline reservation system, the input state might be characterised by particular values of origin, destination, airline, day and flight number. The set of all possible input states is known as the input space.

Similarly, an output variable is a variable that exists external to a program and is set by it. An output state is a set of values of all output variables associated with a run of a program. In the airline reservation system, an output state might be the set of values of variables printed on the ticket and on different reports used in operating the airline. It can now be seen that a departure of the output state from what it is expected to be involves a failure of the program.

The run types required of the program by the environment can be viewed as being selected randomly. Thus, we define the operational profile as the set of run types that the program can execute along with possibilities with which they will occur. In Fig. 7.4, we show two of many possible input states A and B, with their probabilities of occurrence. The part of the operational profile for just these two states is shown in Fig. 7.5. In reality, the number of possible input states is generally quite large. A realistic operational profile is illustrated in Fig. 7.6. Note that the input states have been located on the horizontal axis in order of their probabilities of occurrence. This can be done without loss of generality. They have been placed close together so that the operational profile would appear to be a continuous curve.

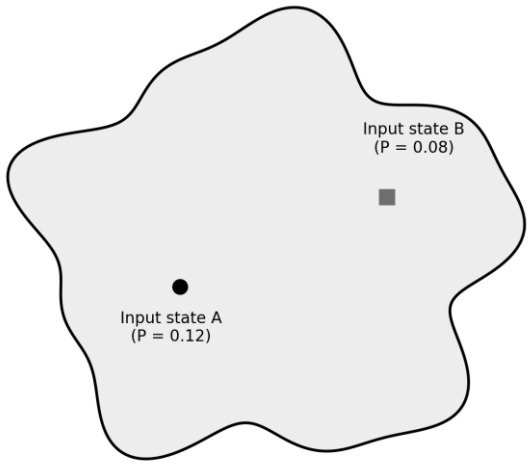


Figure 7.4. The input space, with two input states A and B.

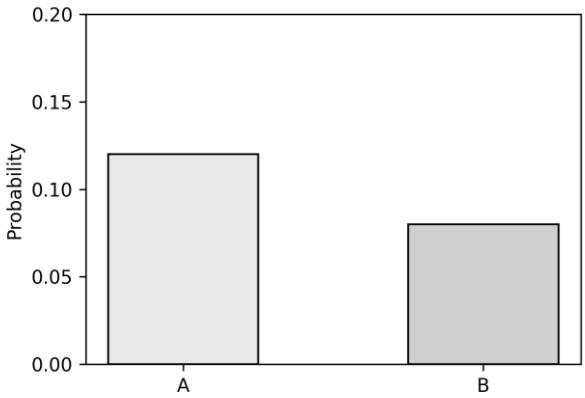


Figure 7.5. The portion of the operational profile for states A and B.

To read the two figures together: the cloud in Fig. 7.4 represents the input space, that is, all possible input states; the two labelled points A and B are two of those states, with probabilities of occurrence 0.12 and 0.08. Fig. 7.5 extracts exactly these two states and plots their probabilities as bars; this is the part of the operational profile that corresponds to them.

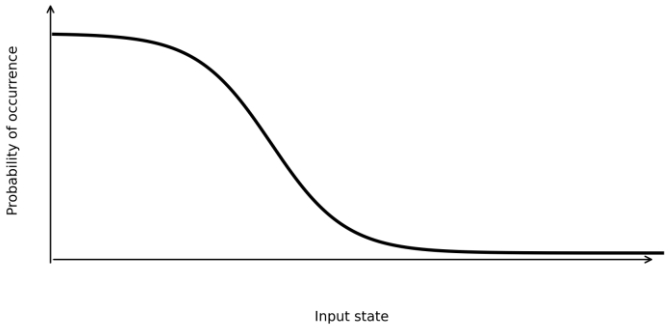


Figure 7.6. Operational profile.

Failure intensity is an alternative way of expressing reliability. We have discussed the example of the reliability of a particular system being 0.95 for 10 hr of time. An equivalent statement in terms of failure intensity can also be made. Fig. 7.7 shows how failure intensity and reliability typically vary during a test period, as faults are removed.

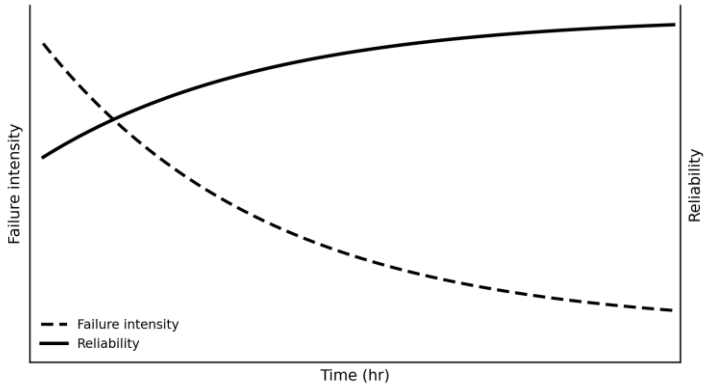


Figure 7.7. Reliability and failure intensity.

7.1.5 Uses of Reliability Studies

There are at least four other ways in which software reliability measures can be of great value to the software engineer, manager or user.

1. You can use software reliability measures to evaluate software engineering technology quantitatively.
2. Software reliability measures offer you the possibility of evaluating development status during the test phases of a project.
3. One can use software reliability measures to monitor the operational performance of software and to control new features added and design changes made to the software.
4. A quantitative understanding of software quality and the various factors influencing it and affected by it enriches the software product and the software development process.

7.1.6 Reliability at the System Level

Reliability is a complex concept, and it should always be considered at the system level rather than at the level of an individual component. The reason is that the components of a system are interdependent: a failure in one component can be propagated through the system and affect the operation of the other components. A defective module may corrupt a shared data structure, an exhausted buffer may stall a communication path, and a wrong return value may travel through several layers before it becomes visible to the user. In a computer-based system, therefore, we have to consider three dimensions when we specify the overall system reliability.

The first dimension is **hardware reliability**. What is the probability of a hardware component failing, and how long does it take to repair that component? The second dimension is **software reliability**. How likely is it that a software component will produce an incorrect output? Software failures are different from hardware failures in that software does not wear out. It can continue operating correctly after an incorrect result has been produced, and the same copy of the program can produce a correct result on the next invocation. The third dimension is **operator reliability**. How likely is it that the operator of a system will make an error? An operator may enter a wrong command, ignore a warning, or apply a correct procedure at the wrong moment, and the resulting behaviour of the system is indistinguishable, from the user's point of view, from a software failure.

The reliability of a system depends upon a number of factors rather than upon the sum of all the probabilities of failure of each factor. A simple additive expression that is often used to indicate the growth of failure sources is:

PS = PA + PR + ... + PN

where PS is the probability of system failure, PA is the probability of failure of component A, PB is the probability of failure of component B, and PN is the probability of failure of component N. As the number of factors increases, the overall probability of system failure increases. This is why a system assembled from individually reliable components can still be unreliable: every additional component, every additional interface, and every additional operator action adds a term to the sum.

Strictly speaking, the additive form is a first approximation rather than an exact probability law. The terms are not mutually exclusive, and for independent components the probability that at least one of them fails is 1 - (1 - PA)(1 - PB)...(1 - PN), which is always smaller than the arithmetic sum. The additive expression is nevertheless useful because it captures the engineering message: reliability is a property of the whole configuration, and it degrades as the number of interacting parts grows. It also explains why reliability engineering prefers simple architectures with few interfaces, why fault containment is designed into critical systems, and why the reliability of a system cannot be inferred from the reliability of its best component.

7.1.7 Reliability Terminologies

The vocabulary of reliability is often used loosely, and the same word is frequently used for different things in different documents. Before any measurement or prediction is attempted, four terms must be fixed precisely, because every later calculation depends on them.

A **system failure** is an event that occurs at some point in time when the system does not deliver a service as expected by its users. A **system error** is erroneous system behaviour, that is, behaviour of the system that does not conform to its specification. A **system fault** is an incorrect system state, that is, a system state that is unexpected by the designers of the system. A **human error or mistake** is human behaviour that results in the introduction of faults into a system.

The four terms form a chain rather than a list. A human error introduces a fault into the program text or into its data; the fault remains dormant until the faulty instruction is executed with a combination of inputs that exposes it; the execution then produces an erroneous state, and the erroneous state, when it reaches the system boundary, becomes a failure. A single fault can be the source of more than one failure, and a single failure can be produced by more than one fault acting together. This is why counting faults is not the same as counting failures, and why a program with many faults in rarely executed code may exhibit few failures in practice.

7.1.8 Classification of Failures

When reliability is specified, the engineer should identify the failure types and consider whether each type should be treated differently in the specification. Many large systems are divided into small subsystems, and each subsystem has different reliability requirements; it is easy and cheap to assess the reliability requirements of each subsystem separately. Failures are commonly classified along several independent axes.

- **Recoverable**: the system can recover, with or without operator intervention.
- **Unrecoverable**: the system cannot recover without operator intervention, or the system may need to be restarted.
- **Transient**: the failure occurs only for certain input values while a function of the system is invoked.
- **Permanent**: the failure occurs for all input values while the same function is invoked.
- **Corrupting**: the failure corrupts system data.
- **Non-corrupting**: the failure does not corrupt system data.
- **Cosmetic**: the failure causes only minor irritation and does not lead to incorrect results, as when a mouse button has to be clicked twice instead of once to invoke a function through the graphical user interface.

The classes are not mutually exclusive. A failure may be transient and corrupting at the same time, or permanent and recoverable, because the classification is based on widely different sets of criteria. The practical value of the classification is that the consequences of failures differ enormously. A transient cosmetic failure is a minor irritant, while an unrecoverable corrupting failure may destroy a database or endanger a patient. Any metric that counts failures without distinguishing their severity therefore describes only half of the reliability problem.

7.1.9 Reliability Metrics

The reliability requirements for different categories of software products may be different. For this reason, the level of reliability required for a software product should be specified in the software requirements specification. In order to do this, we need metrics to express the reliability of a software product quantitatively. A good reliability measure should be observer-independent, so that different people can agree on the degree of reliability a system has. In practice it is very difficult to formulate a metric using which precise reliability measurement would be possible, and the metrics that are used correlate with reliability rather than define it exactly.

Mean time to failure (MTTF). MTTF is the mean time for which a component is expected to be operational, or the average time between observed system failures. An MTTF of 500 means that one failure can be expected every 500 time units. The time units depend entirely on the system and can even be expressed in the number of transactions, as is the case for database query systems. MTTF is relevant for systems with long transactions, that is, where system processing takes a long time, and it should be longer than the transaction length. It is therefore suitable for computer-aided design systems, in which a designer works on a design for several hours, as well as for word-processing systems.

Mean time to repair (MTTR). Once a failure occurs, some time is required to fix the error. MTTR measures the average time it takes to track down the errors causing the failure and then to fix them, or, in hardware terms, the average time to replace a defective component.

Mean time between failures (MTBF). MTTF and MTTR can be combined to obtain the MTBF:

MTBF = MTTF + MTTR

Thus, an MTBF of 300 hours indicates that once a failure occurs, the next failure is expected after 300 hours. In this case the time measurements are real time and not execution time, as they are in MTTF. It is important to note that only run time is considered in MTTF measurements: the time for which the system

is down to fix an error, the boot time and similar intervals are not taken into account, and the clock is stopped at these times.

Probability of failure on demand (POFOD). Unlike the other metrics, this metric does not explicitly involve time measurements. POFOD measures the likelihood that the system will fail when a service request is made. A POFOD of 0.001 means that one out of every thousand service requests may result in a failure. POFOD is an important measure for safety-critical systems and should be kept as low as possible. It is appropriate for systems that are not required to run continuously but deliver a service when a demand is placed on them, such as a library system that is idle until a book-issue request is made, or an emergency shutdown system in a chemical plant.

Rate of occurrence of failure (ROCOF). ROCOF measures the frequency with which unexpected behaviour is likely to occur. A ROCOF of 2/100 means that two failures are likely to occur in each 100 operational time units. This metric is sometimes called the failure intensity. It is relevant for operating systems and transaction-processing systems where the system has to process a large number of similar requests that are relatively frequent, such as credit-card processing systems and airline-booking systems.

Availability (AVAIL). Availability is the probability that the system is available for use at a given time. An availability of 0.998 means that in every 1000 time units the system is likely to be available for 998 of them. Availability is defined as:

Availability = MTTF / (MTTF + MTTR) x 100%

A system with an MTTF of 500 hours and an MTTR of 5 hours, for example, has an MTBF of 505 hours and an availability of 500/505, that is, 0.9901, or 99.01 percent. The MTBF measure is equally sensitive to MTTF and MTTR, whereas the availability measure is somewhat more sensitive to MTTR. This distinction matters in practice: for a telecommunication switch or an operating system that is supposed to be never down, a long repair time is far more damaging than a slightly shorter time between failures, and the design effort is therefore directed at fast recovery rather than at eliminating the last fault.

All of these metrics suffer from shortcomings. One reason is that the metrics are centred on the probability of occurrence of failures but take no account of the consequences of failures. They do not distinguish the relative severity of different failures. Failures that are transient and whose consequences are not serious are in practice of little concern in the operational use of a software product, while more severe failures may render the system totally unusable. Estimating the reliability of a software product more accurately therefore requires the classification of failure types described above, combined with the metric that is appropriate to the way the product is actually used.

7.1.10 Why Software Reliability is Harder to Measure than Hardware Reliability

It is tempting to treat software reliability as if it were hardware reliability computed on a different medium, but the two problems differ in ways that no metric can remove. Hardware components fail mostly because of wear and tear, whereas software components fail because of bugs. A logic gate may be stuck at 1 or 0, or a resistor may short circuit; to fix a hardware fault one has to replace or repair the failed part. A software product, in contrast, continues to fail until the error is tracked down and either the design or the code is changed. When a hardware part is repaired, its reliability is maintained at the level that existed before the failure; when a software failure is repaired, the reliability may either increase or decrease, because a bug fix may introduce new errors. Hardware reliability study is therefore concerned

with stability, that is, with keeping the inter-failure times constant, whereas the aim of software reliability study is reliability growth, that is, an increase in the inter-failure times.

A second difficulty is that the improvement obtained from a repair depends on where the repaired instruction lies. Experimental analysis of the behaviour of a large number of programs shows that about 90 percent of the execution time of a typical program is spent executing only 10 percent of its instructions. The most-used 10 percent of the instructions is often called the core of the program, and the remaining 90 percent of the statements are called non-core. Removing 60 percent of the product defects from the least-used parts of a system would typically produce only a 3 percent improvement in product reliability. The quantity by which the overall reliability improves due to the correction of a single error therefore depends on how frequently the instruction containing the error is executed. If an error is removed from an instruction that is frequently executed, the improvement is large; if it is removed from a rarely used part of the program, the change may not be noticeable at all.

A third difficulty is that reliability is observer-dependent. Different categories of users of a software product typically execute different functions. In a library automation system, for example, the members use functions such as issuing a book and searching for a book, while the librarian uses functions such as creating a member, creating a book record and deleting a member record. Defects that show up for the librarian may never show up for the members. If the functions used by the members are error-free and the functions used by the librarian contain many bugs, the two categories of users will have very different opinions about the reliability of the same product. The perceived reliability depends on the execution profile of the user, not only on the number of latent defects.

The three difficulties together explain why no entirely satisfactory metric for the reliability of a software product exists. The reliability improvement due to fixing a single bug depends on where the bug is located; the perceived reliability of a product is observer-dependent; and the reliability of a product keeps changing as errors are detected and fixed. Reliability metrics remain useful, but they must be interpreted with these limitations in mind, and they must be tied to a stated environment, a stated time period and a stated operational profile, exactly as the definition of software reliability requires.

7.2 Software Quality

Our objective of software engineering is to produce good quality maintainable software in time and within budget. Here quality is very important. But what do we understand with the term "quality"? It is not easy to define quality. People understand quality, appreciate quality but the term may not be able to clearly express the same. It is like beauty which is very much in the eyes of the beholder.

Different people understand different meanings of quality like:

- conformance to requirements
- fitness for the purpose
- level of satisfaction

If a product is meeting its requirements, we may say it is a good quality product. We are assured with respect to requirements are clearly stated and cannot be misunderstood. Everything is measured and designed to requirements and if it matches, product is a quality product. If a car is product with a maximum speed of 150 km/hour and if it fails to achieve in the field, then it is not meeting the

requirements. If two cars are designed with different style, performance and economy and both are up-to standards set for them, then both are quality cars.

When user uses the product and finds the product fit for its purpose, he/she feels that product is of good quality. If product is meeting user requirements, a feeling of satisfaction may emerge and this satisfaction is nothing but the satisfaction for quality.

In a broad sense, the users views of quality must deal with the product's ease of installation, operational efficiency, and convenience. If the product is easy to handle and we comfortably remember how to use it, then customer satisfaction level may increases.

Quality has many characteristics and some are related to each other. In software, the quality is commonly recognised as "lack of bugs" in the program. If a software has too many functional defects, then, it is not meeting its basic requirement of functionality. This is usually expressed in two ways:

1. Defect rate: number of defects per million lines of source code, per function point or any other unit.
2. Reliability: generally measured as number of failures per 't' hours of operation, mean time to failure or probability of failure free operation in a specified time under specified environment.

When we deal with software quality, a list of attributes is required to be defined that are appropriate for software. There are four attribute domains [DUNN90], which should be defined. These are usually the ones most trusted by the customer:

- Reliability
- Usability
- Maintainability
- Adaptability

These four attribute-domains can be divided into attributes that are more commonly understood by the software community and are given in Fig. 7.8. The details of software quality attributes are given in Table 7.4.

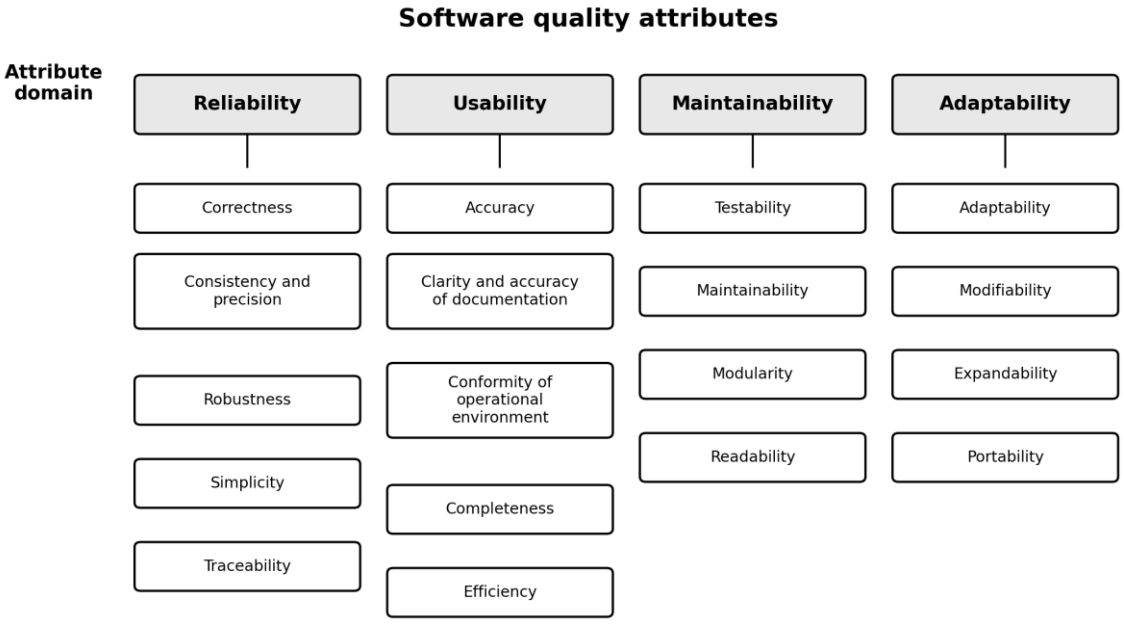


Figure 7.8. Software quality attributes.

Table 7.4: Software quality attributes

Sr. No.	Attribute	Definition
1	Reliability	The extent to which a software performs its intended functions without failure.
2	Correctness	The extent to which a software meets its specifications.
3	Consistency and precision	The extent to which a software is consistent and give results with precision.
4	Robustness	The extent to which a software tolerates the unexpected problems.
5	Simplicity	The extent to which a software is simple in its operations.
6	Traceability	The extent to which an error is traceable in order to fix it.
7	Usability	The extent of effort required to learn, operate and understand the functions of the software.
8	Accuracy	Meeting specifications with precision.
9	Clarity and accuracy of documentation	The extent to which documents are clearly and accurately written.
10	Conformity of operational environment	The extent to which a software is in conformity of operational environment.
11	Completeness	The extent to which a software has specified functions.
12	Efficiency	The amount of computing resources and code required by software to perform a function.
13	Testability	The effort required to test a software to ensure that it performs its intended functions.
14	Maintainability	The effort required to locate and fix an error during maintenance phase.
15	Modularity	It is the extent of ease to implement, test, debug and maintain the software.
16	Readability	The extent to which a software is readable in order to understand.
17	Adaptability	The extent to which a software is adaptable to new platforms and technologies.
18	Modifiability	The effort required to modify a software during maintenance phase.
19	Expandability	The extent to which a software is expandable without undesirable side

		effects.
20	Portability	The effort required to transfer a program from one platform to another platform.

The attribute domain and attributes are also named as the factor and criteria. There are many models of software quality and some are discussed here.

7.2.1 McCall Software Quality Model

McCall et al. [MCCA77] model of software quality was introduced in 1977 and many quality factors were incorporated. The model distinguishes between two levels of quality attributes. Higher level quality attributes are known as quality factors. These are external attributes and can be measured directly. The second level of quality attributes is named as quality criteria. Quality criteria can be measured either subjectively or objectively. The software quality factors are organised in three product quality factors as shown in Fig. 7.9.

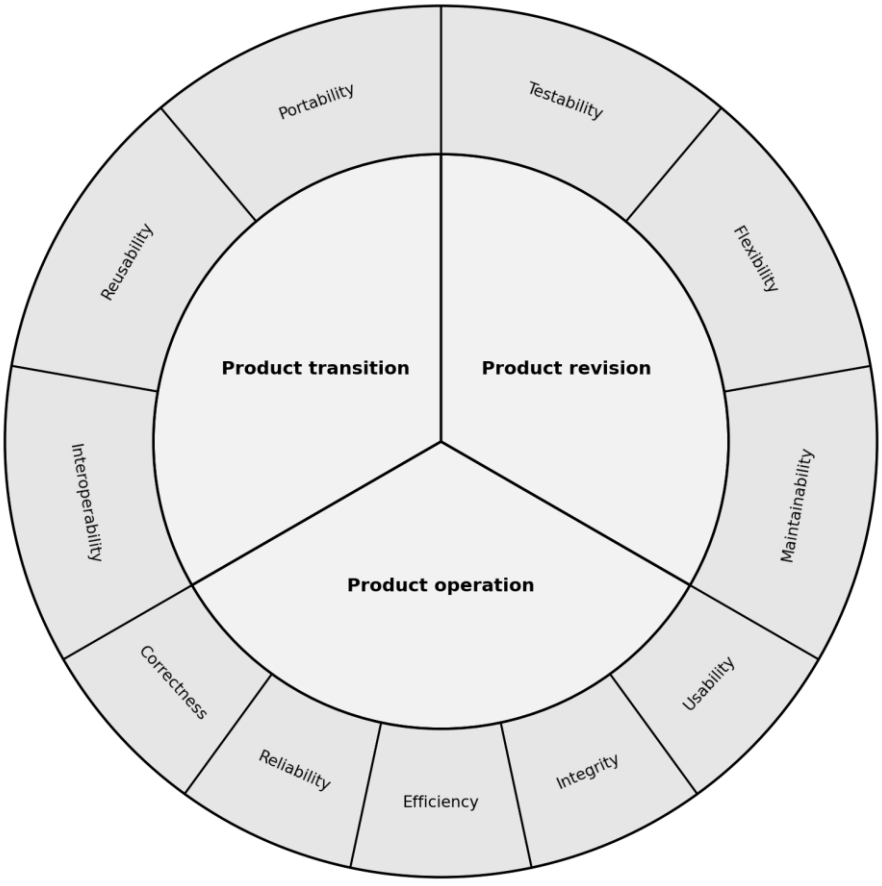


Figure 7.9. Software quality factors.

(i) Product Operation: Here, factors which are related to the operation of a product are combined. The factors are:

- Correctness
- Efficiency
- Integrity
- Reliability
- Usability

These five factors are related to operational performance, convenience, ease of usage and its correctness. These factors play a very significant role in building customer's satisfaction.

(ii) Product Revision: The factors which are required for testing and maintenance are combined and are given below:

- Maintainability
- Flexibility
- Testability

These factors pertain to the testing and maintainability of software. They give us idea about ease of maintenance, flexibility and testing effort. Hence, they are combined under the umbrella of product revision.

(iii) Product Transition: We may have to transfer a product from one platform to another platform or from one technology to another technology. The factors related to such a transfer are combined and are given below:

- Portability
- Reusability
- Interoperability

Most of the quality factors are explained in Table 7.4. The remaining factors are given in Table 7.5.

Table 7.5: Remaining quality factors

Sr. No.	Quality factor	Purpose
1	Integrity	The extent to which access to software or data by the unauthorised persons can be controlled.
2	Flexibility	The effort required to modify an operational program.
3	Reusability	The extent to which a program can be reused in other applications.
4	Interoperability	The effort required to couple one system with another.

Quality criteria

The second level of quality attributes are termed as quality criteria. We have eleven quality factors and each quality factor has many second level of quality attributes which are shown in Fig. 7.10. Second level quality attributes are internal attributes. However, users and managers are interested in the higher level external quality attributes. For example, we may not directly measure the reliability of a software system. We may however, directly measure the number of defects encountered so far. This direct measure can be used to obtain insight into the reliability of the system. This involves a theory of how the number of defects encountered relates to reliability, which can be ascertained on good grounds. For most other aspects of quality though, the relation between the attributes that can be measured directly and the external attributes we are interested in is less obvious, to say the least [VLIE 02]. The relationship

between quality factors and quality criteria is given in Fig. 7.10. The definitions and details of these quality criteria are discussed in Table 7.5(b).

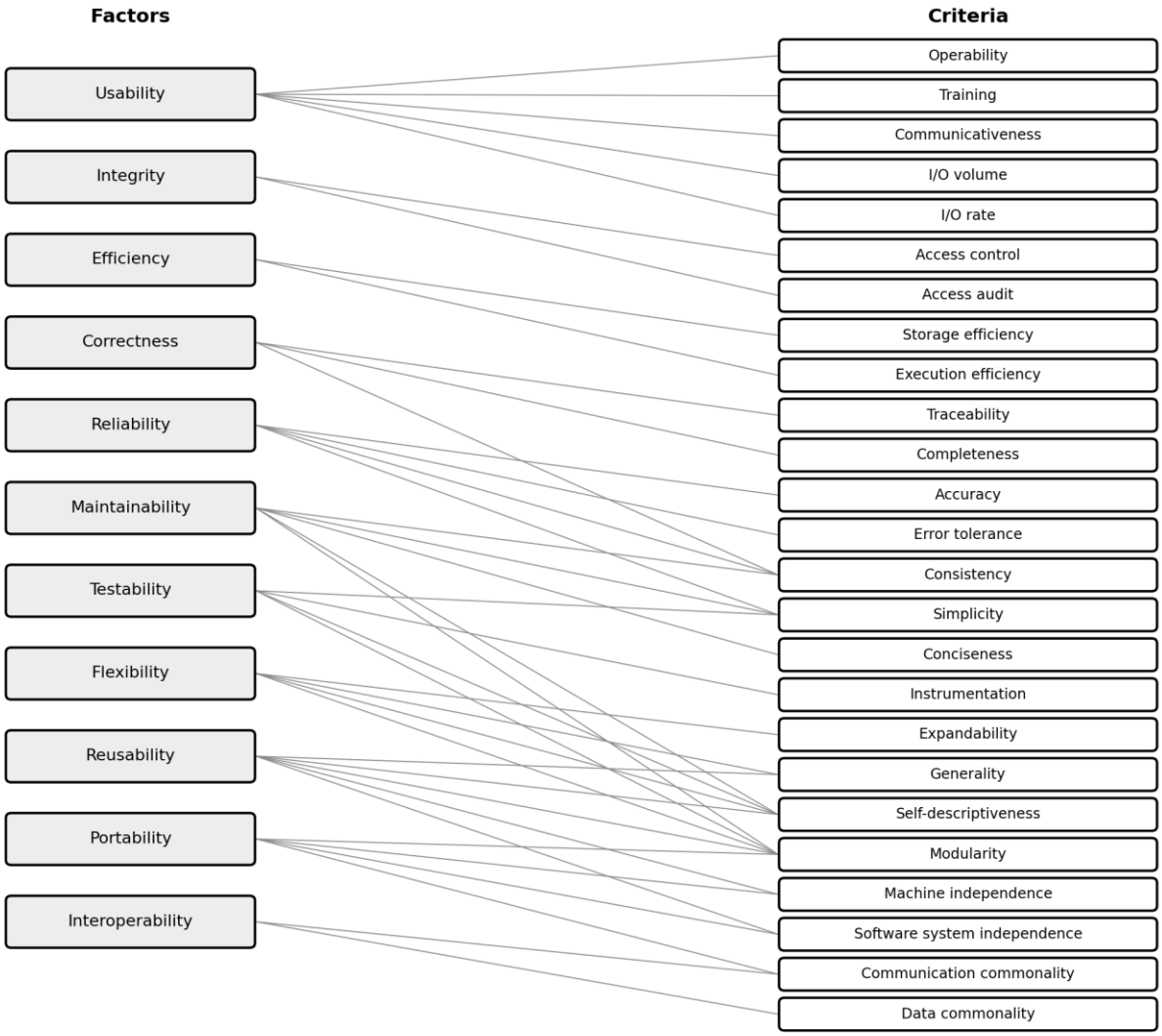


Figure 7.10. McCall's quality model: the relation between quality factors and quality criteria.

Table 7.5(b): Software quality criteria

Sr. No.	Quality criteria	Definition / Purpose
1	Operability	The ease of operation of the software.
2	Training	The ease with which new users can use the system.
3	Communicativeness	The ease with which inputs and outputs can be assimilated.
4	I/O volume	It is related to the I/O volume.
5	I/O rate	It is the indication of I/O rate.
6	Access control	The provisions for control and protection of the software and data.
7	Access audit	The ease with which software and data can be checked for compliance with

		standards or other requirements.
8	Storage efficiency	The run-time storage requirements of the software.
9	Execution efficiency	The run-time efficiency of the software.
10	Traceability	The ability to link software components to requirements.
11	Completeness	The degree to which a full implementation of the required functionality has been achieved.
12	Accuracy	The precision of computations and output.
13	Error tolerance	The degree to which continuity of operation is ensured under adverse conditions.
14	Consistency	The use of uniform design and implementation techniques and notations throughout a project.
15	Simplicity	The ease with which the software can be understood.
16	Conciseness	The compactness of the source code, in terms of lines of code.
17	Instrumentation	The degree to which the software provides for measurements of its use or identification of errors.
18	Expandability	The degree to which storage requirements or software functions can be expanded.
19	Generality	The breadth of the potential application of software components.
20	Self-descriptiveness	The degree to which the documents are self-explanatory.
21	Modularity	The provision of highly independent modules.
22	Machine independence	The degree to which software is dependent on its associated hardware.
23	Software system independence	The degree to which software is independent of its environment.
24	Communication commonality	The degree to which standard protocols and interfaces are used.
25	Data commonality	The use of standard data representations.

It is not easy to measure many quality factors. We may have to apply software metrics, if possible, to measure such factors. The quality factors are not independent, but may overlap. Some factors may impact others in a positive sense, while others may do so negatively. We may have to study and understand such relationships very carefully. A subjective assessment of some criteria can be obtained by giving a rating on a scale from, say, 0 (extremely bad) to 10 (extremely good). Such a subjective metric is difficult to use. Different people assessing the same criterion are likely to give different ratings. This renders a proper quality assessment almost impossible.

There are other methods to assessing quality like decomposing criterion into objectively measurable properties of the system. It is also difficult to measure every property objectively but aim is to define correctly and measure effectively.

7.2.2 Boehm Software Quality Model

Boehm introduced his quality model in 1978 [BOEH78]. He has defined three levels for quality attributes which are given in Fig. 7.11. These levels are primary uses, intermediate constructs and primitive constructs. Intermediate constructs are similar to McCall's quality factors and primitive constructs are similar to quality criteria. Hence Boehm's model is similar to McCall's model in that it presents a hierarchy of characteristics, each of which contributes to overall quality. Boehm's notion of successful software includes the needs and expectations of users, as does McCall's, however, it also includes characteristics of hardware performance that are missing in McCall's model [PFLE 02].

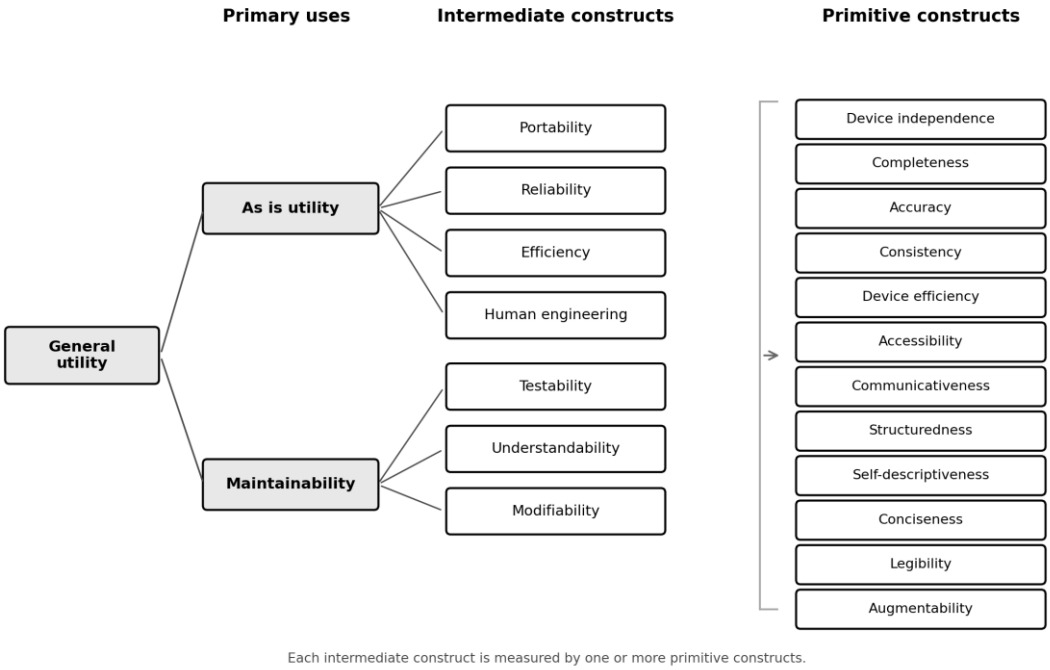


Figure 7.11. The Boehm software quality model.

The users must find the system easy to use. Thus human engineering aspect can sometime be the most critical. A system may be very good from performance point of view, but if user cannot understand how to use it, the system is a failure. The Boehm's model asserts that quality software is software that satisfies the needs of the users. It reflects an understanding of quality where the software:

- does what the user wants it to do
- uses resources correctly and efficiently
- is easy for the user to learn and use
- is well designed, well coded, easily tested and maintained.

7.2.3 ISO 9126

The software engineering community was in the search of a single model to standardise the quality factors since 1980. The advantage of such a universal model is quite obvious; it makes easier to compare one product with another product. The result was ISO 9126 in 1992 as a hierarchical model with six major attributes contributing to quality. These attributes are:

- Functionality
- Reliability
- Usability
- Efficiency
- Maintainability
- Portability

The standard claims that these six factors are comprehensive; that is, any component of software quality can be described in terms of some aspect of one or more of the six factors.

Table 7.6: Software quality characteristics and attributes — The ISO 9126 view

Characteristic / Attribute	Short description of the characteristics and the concerns addressed by attributes
Functionality	Characteristics relating to achievement of the basic purpose for which the software is being engineered
Suitability	The presence and appropriateness of a set of functions for specified tasks
Accuracy	The provision of right or agreed results or effects
Interoperability	Software's ability to interact with specified systems
Security	Ability to prevent unauthorized access, whether accidental or deliberate, to programs and data
Reliability	Characteristics relating to capability of software to maintain its level of performance under stated conditions for a stated period of time
Maturity	Attributes of software that bear on the frequency of failure by faults in the software
Fault tolerance	Ability to maintain a specified level of performance in cases of software faults or unexpected inputs
Recoverability	Capability and effort needed to reestablish level of performance and recover affected data after possible failure
Usability	Characteristics relating to the effort needed for use, and on the individual assessment of such use, by a stated implied set of users
Understandability	The effort required for a user to recognize the logical concept and its applicability
Learnability	The effort required for a user to learn its application, operation, input, and output
Operability	The ease of operation and control by users
Efficiency	Characteristic related to the relationship between the level of performance of the software and the amount of resources used, under stated conditions
Time behaviour	The speed of response and processing times and throughput rates in performing its function
Resource behaviour	The amount of resources used and the duration of such use in performing its function
Maintainability	Characteristics related to the effort needed to make modifications, including corrections, improvements or adaptation of software to changes in environment, requirements and functional specifications
Analyzability	The effort needed for diagnosis of deficiencies or causes of failures, or for identification of parts to be modified
Changeability	The effort needed for modification, fault removal or for environmental change
Stability	The risk of unexpected effect of modifications
Testability	The effort needed for validating the modified software
Portability	Characteristics related to the ability to transfer the software from one organization or hardware or software environment to another
Adaptability	The opportunity for its adaptation to different specified environments
Installability	The effort needed to install the software in a specified environment
Conformance	The extent to which it adheres to standards or conventions relating to portability
Replaceability	The opportunity and effort of using it in the place of other software in a particular environment

The hierarchical model is given in Fig. 7.12. The software quality attributes are explained in Table 7.6.

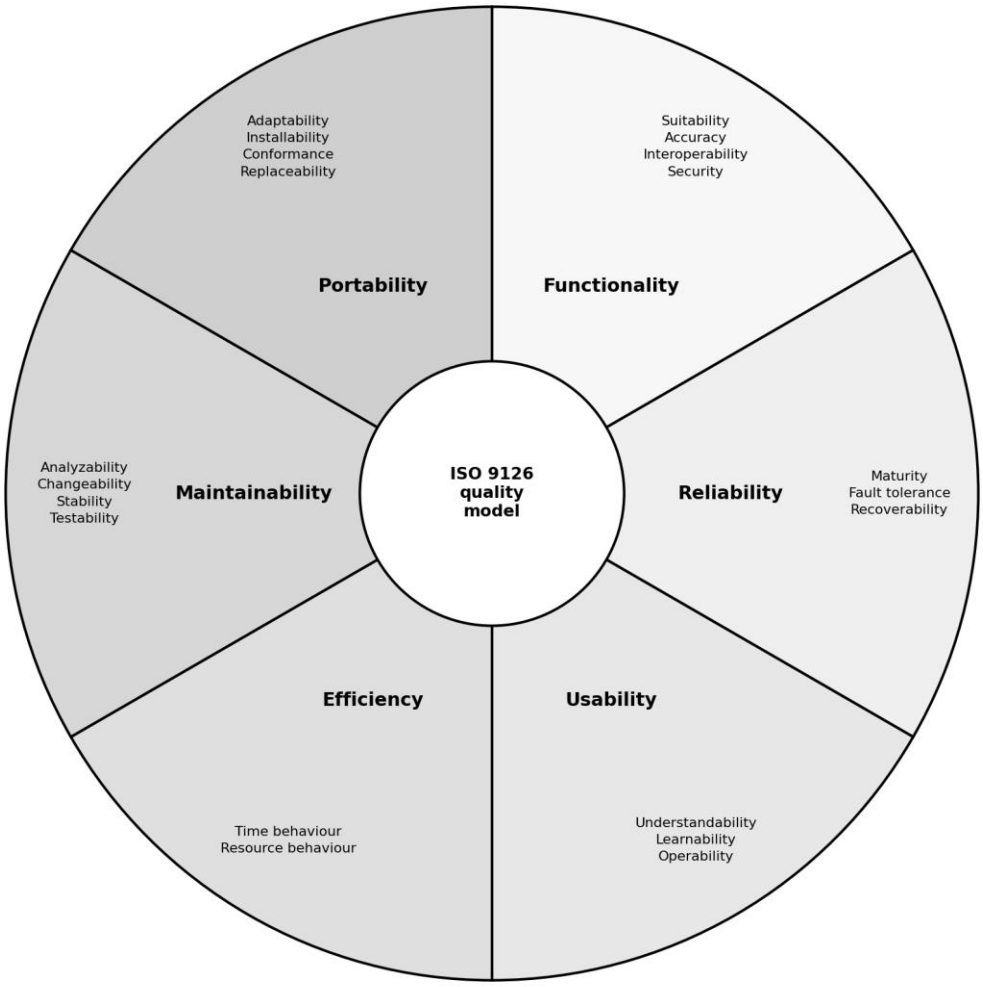


Figure 7.12. ISO 9126 quality model.

One major difference between the ISO model and those of McCall and Boehm is that the ISO hierarchy is strict. Each characteristic is related only to one attribute. Moreover, outer attributes are related to the user view of the software rather than to an internal, developer view.

These models are helpful in articulating just what it is that we value in the software we build and use. But there are a number of difficulties in the direct application of any of the above models. We have defined quality as "meeting requirements", and it is not possible from a practical point of view to define one generic model which can fit into all types of application domain.

These models take for granted that all high-level quality attributes are independent of each other. Based on this assumption, they have decomposed higher-level quality factors into lower-level ones independently [NIHA 01]. Thus, design of a generic model is not easy due to the various attributes of quality. Sometimes quality is difficult to express, although easy to feel and experience. These problems also make it difficult for us to determine how much a given model is effective and complete.

7.2.4 The Quality Factors in Detail

The eleven factors of the McCall model are not merely labels for sections of a report; each one names a property that can be argued about, designed for, and to some extent measured. The definitions that follow make the content of the model concrete.

Correctness is the extent to which a program satisfies its specification and fulfils the customer's mission objectives. Correctness is the most basic of the factors, because a program that does not do what it is supposed to do has no quality at all; but it is also narrower than reliability, since it says nothing about how the program behaves under conditions that the specification did not anticipate.

Reliability is the extent to which a program can be expected to perform its intended function with the required precision. The word expected is important here: reliability is a statement about future behaviour under stated conditions, and it cannot be observed directly in a single execution.

Efficiency is the amount of computing resources and code required by a program to perform its function. It includes both the run-time resources consumed and the storage occupied, and it is the factor that most directly conflicts with the others: a highly efficient program is often less portable and less maintainable than a straightforward one.

Integrity is the extent to which access to software or data by unauthorized persons can be controlled. In modern terms it covers authentication, authorization and audit, and it is the ancestor of the security characteristic of later models.

Usability is the effort required to learn, operate, prepare input for, and interpret the output of a program. It is a property of the interface between the program and its user rather than of the program text, which is why it can be assessed only with the participation of representative users.

Maintainability is the effort required to locate and fix an error in a program. This is a deliberately narrow definition: the wider sense of maintenance includes enhancement, adaptation and retesting, and all of these activities depend on the structure of the program rather than on any single measure.

Flexibility is the effort required to modify an operational program, that is, to change behaviour that is already in service. A flexible program can absorb new requirements without a redesign of its architecture.

Testability is the effort required to test a program to ensure that it performs its intended function. Testability is not the same as correctness: a program may be correct and still be almost impossible to test, because its control flow is tangled, its outputs are not observable, or its state cannot be set to a known condition.

Portability is the effort required to transfer the program from one hardware and/or software system environment to another. It depends on the number and the depth of the assumptions that the program makes about its platform.

Reusability is the extent to which a program, or parts of a program, can be reused in other applications. It is related to the packaging and the scope of the functions that the program performs: a component that solves a general problem in a well-defined interface is reusable, while a fragment that is entangled with the rest of an application is not.

Interoperability is the effort required to couple one system to another. It covers the use of standard interfaces, protocols and data representations, and it grows in importance as systems are assembled from components rather than written from scratch.

The three groups into which these factors are organised are not arbitrary. Product operation groups the factors that describe how well the product works while it is running. Product revision groups the factors that describe how easily the product can be changed. Product transition groups the factors that describe how easily the product can be moved to another environment. The grouping reflects the three different questions that users, maintainers and integrators ask about the same product, and it explains why the same program can be judged excellent by one group and poor by another.

7.2.5 Measuring the Quality Factors

It is difficult, and in some cases impossible, to develop direct measures of the quality factors. We therefore define a set of metrics and use them to develop expressions for each factor according to the relationship

$$Fq = c1\ m1 + c2\ m2 + ... + cn\ mn$$

where Fq is a software quality factor, the cn are regression coefficients, and the mn are the metrics that affect the quality factor. The coefficients express the relative weight of each metric for a particular factor, and they are obtained by regression against the ratings that experienced engineers assign to a set of past products. In principle, the same equation can be calibrated differently for different application domains, because the relative importance of, say, efficiency and portability is not the same in a compiler and in a payroll system.

Unfortunately, many of the metrics defined by McCall and his colleagues can be measured only subjectively. The metrics may take the form of a checklist that is used to grade specific attributes of the software, and the grading scheme is a scale from 0, low, to 10, high. The metrics used in the grading scheme include auditability, the ease with which conformance to standards can be checked; accuracy, the precision of computations and control; communication commonality, the degree to which standard interfaces, protocols and bandwidth are used; completeness, the degree to which full implementation of the required function has been achieved; conciseness, the compactness of the program in terms of lines of code; consistency, the use of uniform design and documentation techniques throughout the project; data commonality, the use of standard data structures and types throughout the program; error tolerance, the damage that occurs when the program encounters an error; execution efficiency, the run-time performance of the program; expandability, the degree to which architectural, data or procedural design can be extended; generality, the breadth of potential application of program components; hardware independence, the degree to which the software is decoupled from the hardware on which it operates; instrumentation, the degree to which the program monitors its own operation and identifies errors; modularity, the provision of highly independent modules; operability, the ease of operation of the software; security, the availability of mechanisms for the control and protection of the program and its data; self-documentation, the degree to which the source code provides meaningful documentation; simplicity, the ease with which the program can be understood; software system independence, the degree to which the program is independent of its environment; training, the ease with which new users can use the system; and traceability, the ability to link software components to requirements.

A subjective assessment of some criteria can be obtained by giving a rating on a scale from, say, 0, extremely bad, to 10, extremely good. Such a subjective metric is difficult to use. Different people assessing the same criterion are likely to give different ratings, and the same person may give different ratings on different days. This renders a proper quality assessment almost impossible, and it is the main reason why the models of McCall, Boehm and ISO are used as frameworks for discussion and for the selection of attributes rather than as instruments for the precise calculation of a quality score.

There are other methods of assessing quality, such as decomposing a criterion into objectively measurable properties of the system. It is also difficult to measure every property objectively, but the aim is to define the properties correctly and to measure them effectively. The practical compromise is to combine a small number of objective measures, such as defect density or mean time between failures, with a structured expert assessment of the attributes that resist measurement.

7.2.6 Software Quality Assurance

Software quality assurance, usually abbreviated SQA, has emerged as one of the most discussed topics in the software industry. The major aim of SQA is to help an organisation develop high quality software products in a repeatable manner. A software development organisation can be called repeatable when its software development process is person-independent, that is, when the success of a project does not depend on exactly who the members of the team happen to be. Besides quality, SQA also addresses the cost of development, because quality that is achieved by unbounded testing is not quality that an organisation can sustain.

SQA is often confused with quality control, and the distinction is worth stating precisely. Quality control is the set of activities that examine the product after it has been produced and reject the items that do not meet the standard; testing, inspection and review are quality-control activities. Quality assurance is the set of activities that are designed to prevent defects from being introduced in the first place, and it therefore operates on the process rather than on the product. The distinction mirrors the one between inspecting the parts at the end of a production line and designing the production line so that defective parts are not produced.

The activities that make up an SQA programme can be grouped into a small number of families. The first is the definition and enforcement of standards: coding standards, documentation standards, design notations, and the standards for the review and change of work products. The second is the conduct of formal technical reviews, in which the work product is examined by colleagues before it is passed to the next phase. The third is testing, at the unit, integration, system and acceptance levels, with the results recorded and traced to requirements. The fourth is configuration management, which ensures that the version of a component that is tested is the version that is delivered, and that changes are authorized and recorded. The fifth is measurement: the collection of defect data, effort data and failure data, without which no process can be improved in a controlled way.

These activities are documented in an SQA plan, which identifies the standards that apply to the project, the reviews and audits that will be conducted, the measurements that will be collected, and the organisational responsibility for each of them. The effectiveness of the plan depends heavily on the independence of the group that executes it. If the same team that writes the code also decides whether the code is acceptable, the review becomes a formality; for this reason, in mature organisations the SQA

function reports outside the project team, and the results of audits are visible to management rather than only to the developers.

7.2.7 Quality Metrics in Practice

The models describe what quality is; metrics describe how much of it a particular product has. A small set of metrics is sufficient to give management a usable picture, provided that each metric is defined precisely and that its limitations are understood.

Table 7.7: Commonly used software quality metrics

Metric	Definition	Interpretation
Defect density	Number of defects found per thousand lines of code or per function point	Normalises the defect count by size, so that products of different sizes can be compared; useful for tracking a product across releases.
Defect removal efficiency	DRE = E / (E + D), where E is the number of defects found before delivery and D the number found after delivery	Measures the effectiveness of the verification activities; a value close to 1 means that few defects escape to the customer.
Mean time between failures	MTBF = MTTF + MTTR	Expresses reliability in the units that the user experiences; an increasing MTBF across releases is evidence of reliability growth.
Availability	MTTF / (MTTF + MTTR) x 100%	Captures both failure frequency and repair time; the appropriate measure for systems that must be continuously available.
Probability of failure on demand	Proportion of service requests that result in a failure	Appropriate for systems that run on demand rather than continuously, and for safety-related functions.
Rate of occurrence of failure	Number of failures per unit of operational time	Appropriate for systems that process a continuous stream of similar requests.
Rework effort	Effort spent correcting work that was already completed, as a percentage of total effort	A process indicator: high rework indicates that defects are being found late or that requirements are unstable.
Customer-reported defects	Number of defects reported by users per unit of time or per installation	The ultimate measure of quality in operation, and the one least under the control of the development team.

Two cautions apply to the whole table. First, a metric is meaningful only in comparison: with a previous release, with a similar product, or with a target that was set in advance. An absolute defect density of two defects per thousand lines says nothing by itself, because the acceptable value depends on the domain, on the language and on the age of the code. Second, metrics interact. A team that is rewarded for a low defect count can achieve it by classifying defects differently or by deferring the reporting of failures, and a team that is rewarded for a short schedule can achieve it by reducing review effort, which shifts cost into the failure column. For this reason, quality metrics are used in combination, and they are interpreted together with the process data that produced them.

7.2.8 Beyond ISO 9126: the ISO 25010 View

The ISO 9126 model was revised and replaced by the ISO/IEC 25010 standard, which is part of the SQuaRE family of standards for software product quality. The revision kept the idea of a hierarchy of characteristics and sub-characteristics, but it changed the top level of the hierarchy in ways that reflect the experience of the intervening decade.

The most visible change is the separation of functionality into functional suitability, and the addition of two characteristics that the earlier model treated inside other categories. Compatibility collects the properties that concern the coexistence of the product with other products in a shared environment, and security collects the properties that concern the protection of information and of access. Performance efficiency replaces the older efficiency characteristic, and it distinguishes time behaviour from resource utilization and capacity. The result is a model with eight characteristics instead of six.

Table 7.8: From ISO 9126 to ISO 25010

ISO 9126 characteristic	ISO 25010 characteristic	Nature of the change
Functionality	Functional suitability	Renamed; sub-characteristics become functional completeness, functional correctness and functional appropriateness.
Reliability	Reliability	Retained; sub-characteristics become maturity, availability, fault tolerance and recoverability, so that availability is now explicitly part of reliability.
Usability	Usability	Retained and extended with user error protection, user interface aesthetics and accessibility.
Efficiency	Performance efficiency	Renamed; time behaviour, resource utilization and capacity.
Maintainability	Maintainability	Extended with modularity and reusability, which were separate factors in the McCall model.
Portability	Portability	Retained; adaptability, installability and replaceability.
Not a separate characteristic	Compatibility	New; co-existence and interoperability.
Not a separate characteristic	Security	New at the top level; confidentiality, integrity, non-repudiation, accountability and authenticity.

For the purposes of this chapter the difference between the standards is less important than the continuity of the underlying idea. Both models assert that quality is a hierarchy, that the higher-level attributes are the ones that users care about, and that the lower-level attributes are the ones that developers can influence. Both models accept that the attributes overlap, that the boundaries between them are conventional, and that no single metric can capture the whole. The vocabulary changes; the engineering problem does not.

The practical consequence for a project is that the choice of a quality model should be made early, recorded in the quality plan, and used consistently in requirements, in design reviews and in acceptance criteria. A model that is adopted half-way through a project, or that is quoted in a report but never used to make a decision, adds vocabulary without adding quality.

7.2.9 The FURPS Quality Model

The quality factors described by McCall and his colleagues represent one of a number of suggested checklists for software quality. Hewlett-Packard developed another set of software quality factors that has been given the acronym FURPS: functionality, usability, reliability, performance, and supportability. The FURPS quality factors draw liberally from earlier work and define the following attributes for each of the five major factors.

Functionality is assessed by evaluating the feature set and capabilities of the program, the generality of the functions that are delivered, and the security of the overall system.

Usability is assessed by considering human factors, overall aesthetics, consistency, and documentation.

Reliability is evaluated by measuring the frequency and severity of failure, the accuracy of output results, the mean time to failure, the ability to recover from failure, and the predictability of the program.

Performance is measured by processing speed, response time, resource consumption, throughput, and efficiency.

Supportability combines the ability to extend the program, that is, extensibility, adaptability and maintainability, with the other attributes that make a product supportable: testability, compatibility, configurability, the ease with which a system can be installed, and the ease with which it can be localized.

Table 7.15: The FURPS quality factors

Factor	Attributes used in its assessment
Functionality	Feature set and capabilities; generality of the functions; security of the system
Usability	Human factors; overall aesthetics; consistency; documentation
Reliability	Frequency and severity of failure; accuracy of output; mean time to failure; recoverability; predictability
Performance	Processing speed; response time; resource consumption; throughput; efficiency
Supportability	Extensibility; adaptability; maintainability; testability; compatibility; configurability; installability; localizability

The FURPS model is less detailed than the McCall model, and it is deliberately so. Its five factors are the categories that a customer or a product manager is likely to name when asked what quality means for a particular product, and the attributes listed under each factor are the questions that the technical team must answer in order to show that the factor has been achieved. The model is therefore useful at the boundary between requirements and design: it turns a vague demand for a good product into five conversations, each with its own measures.

Comparing FURPS with McCall is instructive. McCall separates operation, revision and transition, and it treats maintainability, flexibility and testability as separate factors. FURPS folds them into supportability, and it promotes performance to a factor of its own. Neither model is more correct than the other; they are different partitions of the same space of concerns, and the choice between them depends on which partition matches the vocabulary of the organization that will use it.

7.2.10 Representative Qualities

Beyond the factor models, it is useful to examine the most important qualities of software products and processes one by one, because each of them has a precise meaning that is often lost when it is used as a label in a table.

Correctness. A program is functionally correct if it behaves according to the specification of the functions it should provide, called the functional requirements specification. It is common simply to use the term correct rather than functionally correct, and the term specification implies the functional requirements specification. The definition of correctness assumes that a specification of the system is available and that it is possible to determine unambiguously whether or not a program meets the specification. With most current software systems, no such specification exists; if a specification does exist, it is usually written in an informal style using natural language.

Reliability. Informally, software is reliable if the user can depend on it. The specialized literature on software reliability defines reliability in terms of statistical behaviour, that is, the probability that the

software will operate as expected over a specified time interval. The relationship between reliability and correctness is one of inclusion: the set of all reliable programs includes the set of correct programs, but not vice versa. A program can be correct, in the sense that it satisfies its specification for the inputs that the specification describes, and still be unreliable, because it fails on inputs that the specification did not describe.

Robustness. A program is robust if it behaves reasonably even in circumstances that were not anticipated in the requirements specification, for example when it encounters incorrect input data or some hardware malfunction such as a disk crash. Robustness is a difficult quality to define: if we could state precisely what we should do to make an application robust, we would be able to specify its reasonable behaviour completely, and robustness would become equivalent to correctness.

Performance. Performance is important because it affects the usability of the system. If a software system is too slow, it reduces the productivity of the users, possibly to the point of not meeting their needs. If it uses too much disk space, it may be too expensive to run. If it uses too much memory, it may affect the other applications that run on the same system, or it may run slowly while the operating system tries to balance the memory usage of the different applications. Performance is also important because it affects the scalability of a software system.

Verifiability. A software system is verifiable if its properties can be verified easily. For example, it is important to be able to verify the correctness or the performance of a software system. Verifiability is usually an internal quality, although it sometimes becomes an external quality as well: in many security-critical applications the customer requires the verifiability of certain properties, and the highest level of the security standard for a trusted computer system requires the verifiability of the operating system kernel.

Repairability. A software system is repairable if it allows the correction of its defects with a limited amount of work. In many engineering products repairability is a major design goal; automobile engines, for example, are built with the parts that are most likely to fail as the most accessible. In computer hardware engineering there is a subspecialty called repairability, availability and serviceability.

Evolvability. Like other engineering products, software products are modified over time to provide new functions or to change existing functions. The fact that software is so malleable makes modifications extremely easy to apply to an implementation, and that ease is precisely what makes evolvability both valuable and dangerous: a change that is cheap to make may still be expensive to verify.

Understandability. Some software systems are easier to understand than others, and some tasks are inherently more complex than others. Given tasks of inherently similar difficulty, we can follow certain guidelines to produce more understandable designs and to write more understandable programs; abstraction and modularity, for example, enhance a system's understandability.

Interoperability. Interoperability refers to the ability of a system to coexist and cooperate with other systems. With interoperability, a vendor can produce different products and allow the user to combine them if necessary. This makes it easier for the vendor to produce the products, and it gives the user more freedom in exactly which functions to pay for and to combine.

The list is useful because it separates qualities that are frequently confused. Correctness is not reliability, reliability is not robustness, and robustness is not performance. A program that is correct on the specified

inputs may still be unreliable on the unspecified ones; a program that is reliable within its specified environment may still be fragile when the environment changes; and a program that is fast may still be unusable. The quality models described earlier group these qualities into factors, but the factors are only as good as the definitions that support them.

7.2.11 The SQA Plan

An SQA plan defines the quality processes and procedures that should be used. This involves selecting and instantiating standards for products and processes and defining the required quality attributes of the system. The SQA plan provides a roadmap for instituting software quality assurance. Developed by the SQA group, or by the software team if an SQA group does not exist, the plan serves as a template for SQA activities that are instituted for each software project. The quality plan should select those organizational standards that are appropriate to a particular product and development process; new standards may have to be defined if the project uses new methods and tools.

An outline structure for a quality plan includes five parts. The first is the product introduction: a description of the product, its intended markets, and the quality expectations for the product. The second is the product plans: the critical release dates and responsibilities for the product, along with plans for distribution and product servicing. The third is the process descriptions: the development and service processes that should be used for product development and management. The fourth is the quality goals: the quality goals and plans for the product, including an identification and justification of critical product quality attributes. The fifth is risks and risk management: the key risks that might affect product quality and the actions to address these risks.

Preparation of a software quality assurance plan for each software project is a primary responsibility of the software quality assurance group. The topics that such a plan normally covers are listed in Table 7.16.

Table 7.16: Topics in a software quality assurance plan

Topic	What the plan states
Purpose and scope	Why the plan exists and which projects, products and phases it covers
References	The other documents to which the plan refers
Management	The organization, the tasks and the responsibilities for quality
Documentation	The documents to be produced and the standards they must meet
Standards, practices and conventions	The technical standards that the project must follow
Reviews and audits	The reviews and audits that will be conducted, and their criteria
Testing	The test strategy, the levels of testing and the records to be kept
Problem reporting and corrective action	How defects are reported, tracked and closed
Tools, techniques and methodologies	The support environment used to enforce the plan
Code, media and supplier control	How code, data media and subcontracted items are controlled
Records collection, maintenance and retention	Which records are kept, for how long, and where
Training	The training required for the staff to execute the plan
Risk management	The methods of risk management that are to be used

The plan is a working document, not an certificate. Its value is measured by whether a reviewer can take it, open a project file, and determine within a few minutes whether the project is being run in the way that the plan requires. A plan that lists standards which nobody applies, or reviews which leave no record, is worse than no plan at all, because it creates the appearance of control without the substance.

7.2.12 Classification of Software Qualities

There are many desirable software qualities, and some of them apply both to the product and to the process used to produce the product. The user wants the software products to be reliable, efficient, and easy to use. The producer of the software wants it to be verifiable, maintainable, portable, and extensible. The manager of the software project wants the process of software development to be productive and easy to control. Two classifications are useful for organizing this variety: internal versus external, and product versus process.

External versus internal qualities. Software qualities can be divided into external and internal qualities. The external qualities are visible to the users of the system; the internal qualities are those that concern the developers of the system. In general, users of the software only care about the external qualities, but it is the internal qualities, which deal largely with the structure of the software, that help developers achieve the external qualities. For example, the internal quality of verifiability is necessary for achieving the external quality of reliability. In many cases, however, the qualities are related closely and the distinction between internal and external is not sharp.

The distinction matters because it determines who can judge a quality and when. A user can judge response time, ease of learning and freedom from failures, because those qualities are visible at the interface. A user cannot judge modularity or traceability directly, although both may be the reason why the product behaves well. The developer, in contrast, can judge the internal qualities long before the product is delivered, which is why internal qualities are the ones that can be designed and reviewed early.

Product and process qualities. We use a process to produce the software product, and we can also attribute some qualities to the process, although process qualities often are closely related to product qualities. For example, if the process requires careful planning of system test data before any design and development of the system starts, the product is likely to be more reliable. The process quality of early test planning produces the product quality of reliability, and the relationship is causal rather than accidental.

This second classification is the bridge to the maturity models of Section 7.4. A process that is defined, measured and improved is a process with high process quality, and the CMM asserts, level by level, that high process quality is the most reliable route to high product quality. The assertion is not a theorem, but it is supported by the observation that organizations which repeat their successes are those whose processes do not depend on the accident of who happens to be on the team.

7.2.13 Why Software Quality Matters

We would expect quality to be a concern of all producers of goods and services. However, the special characteristics of software, and in particular its intangibility and complexity, make special demands. Three reasons are usually given.

Increasing criticality of software. The final customer or user is naturally anxious about the general quality of software, especially its reliability. This is increasingly the case as organizations become more dependent on their computer systems and as software is used more and more in areas which are safety critical, for example to control aircraft.

The intangibility of software. Intangibility makes it difficult to know whether a particular task in a project has been completed satisfactorily. The results of these tasks can be made tangible by demanding

that the developers produce deliverables that can be examined for quality. The demand for deliverables is not bureaucracy; it is the mechanism by which an invisible product becomes reviewable.

Accumulating errors during software development. As computer system development is made up of a number of steps where the output from one step is the input to the next, an error introduced early is carried forward and may be amplified by the later steps. The earlier a defect is found, the cheaper it is to correct, and the later it is found, the more of the design and the code has been built on the assumption that it is correct.

These three reasons explain why quality cannot be inspected into a product at the end. Criticality raises the cost of failure, intangibility removes the visible evidence that would allow an early judgement, and accumulation means that the evidence arrives too late. A quality programme is therefore an attempt to make the invisible visible early, which is precisely what reviews, standards, measurements and audits are for.

7.2.14 Fitness of Purpose and the Modern View of Quality

Traditionally, the quality of a product is defined in terms of its fitness of purpose. A good quality product does exactly what the users want it to do, and for almost every product fitness of purpose is interpreted in terms of satisfaction of the requirements laid down in the requirements specification. Although fitness of purpose is a satisfactory definition of quality for many products such as a car, a table fan or a grinding machine, it is not a wholly satisfactory definition of quality for software products.

Consider a software product that is functionally correct, that is, it correctly performs all the functions that have been specified in its requirements document. Even though it may be functionally correct, we cannot consider it to be a quality product if it has an almost unusable user interface. Another example is a product which does everything that the users wanted but has almost incomprehensible and unmaintainable code. The traditional concept of quality as fitness of purpose is therefore not wholly satisfactory for software.

Unlike hardware products, software lasts a long time, in the sense that it keeps evolving to accommodate changed circumstances. The modern view of quality associates a software product with several quality factors or attributes, including the following. A product is portable if it can easily be made to work in different hardware and operating system environments and easily interface with external hardware devices and software products. It has good usability if different categories of users, both expert and novice, can easily invoke its functions. It has good reusability if different modules of the product can easily be reused to develop new products. It is correct if the different requirements specified in the requirements document have been correctly implemented. It is maintainable if errors can be easily corrected as and when they show up, new functions can be easily added, and the functionalities can be easily modified.

The modern view therefore replaces a single test, does it do what the specification says, with a set of tests that ask whether the product will remain valuable after delivery. The replacement is not a rejection of correctness; it is a recognition that correctness is necessary but not sufficient. A product that is correct today and impossible to change tomorrow has failed the second test even though it passed the first.

7.2.15 From Quality Factors to Quantitative Measures

The overriding goal of software engineering is to produce a high-quality system, application or product. To achieve this goal, software engineers must apply effective methods coupled with modern tools within the context of a mature software process. In addition, a good software engineer and a good software manager must measure if high quality is to be realized.

The quality of a system, application or product is only as good as the requirements that describe the problem, the design that models the solution, the code that leads to an executable program, and the tests that exercise the software to uncover errors. A good software engineer uses measurement to assess the quality of the analysis and design models, the source code, and the test cases that have been created as the software is engineered. To accomplish this real-time quality assessment, the engineer must use technical measures to evaluate quality in objective rather than subjective ways. The project manager must also evaluate quality as the project progresses: private metrics collected by individual software engineers are assimilated to provide project-level results.

This is the transition from a qualitative to a quantitative view of quality. In the qualitative view, a reviewer asks whether the design is good; in the quantitative view, the reviewer asks how many requirements are traceable to design elements, how many decisions are documented, how many defect reports are open, and how the answers compare with the previous release. The quantitative view is not a claim that every quality can be reduced to a number. It is a claim that the questions which can be made objective should be made objective, because an objective question can be answered consistently by different people on different days.

The transition has a cost. Measures must be defined, collected, validated and interpreted, and a measure that is defined carelessly will be optimized in a way that damages the quality it was meant to describe. The discipline that prevents this is the same one that the quality models provide: state the factor, state the criterion, state the metric, and state the interpretation before the data are collected. A metric without a stated interpretation is a number without a meaning.

7.2.16 Three Views of Quality

Our objective of software engineering is to produce good quality maintainable software in time and within budget. Here quality is very important. But what do we understand by the term quality? It is not easy to define quality. People understand quality and appreciate quality, but the term may not be able to express the same thing clearly. It is like beauty, which is very much in the eyes of the beholder.

Different people understand different meanings of quality, and three of those meanings are common enough to be worth separating.

Conformance to requirements. In this view a product is good if it does what the specification says. The view has the great advantage of being checkable: a requirement can be tested, and a test can pass or fail. Its weakness is that it says nothing about the requirements themselves. A product that conforms perfectly to a poor specification is a poor product, and the view provides no way of noticing.

Fitness for the purpose. In this view a product is good if it does what the user needs, whether or not the need was written down. The view corrects the weakness of the first, and it introduces a new difficulty: the purpose must be known, and the purpose is often discovered only after the product has been used. Fitness for purpose is the view that underlies the modern quality factors, and it is the reason why usability,

maintainability and portability appear in the models even though they are rarely stated in a requirements document.

Level of satisfaction. In this view quality is the degree to which the product satisfies the people who use it and the people who pay for it. The view is broader than the other two, and it is the one that recognises that quality is experienced rather than measured. Its weakness is that satisfaction is subjective and can be influenced by expectations, by price and by the alternatives that are available.

The three views are not competitors; they are three levels of the same question. Conformance asks whether we built the product right, fitness asks whether we built the right product, and satisfaction asks whether the product made a difference. A quality programme that measures only the first will deliver a product that passes its tests and disappoints its users, and a programme that measures only the third will have no way of knowing why the users are disappointed. The quality models of this chapter are, in effect, an attempt to make the second and third views as checkable as the first.

7.3 Software Reliability Models

To model software reliability one must first consider the principal factors that affect it: fault introduction, fault removal, and the environment. Fault introduction depends primarily on the characteristics of the developed code, that is, the code created or modified for the application, and on the characteristics of the development process, which include the software engineering technologies and tools used and the level of experience of the personnel. Note that code can be developed to add features or to remove faults. Fault removal depends upon time, the operational profile, and the quality of the repair activity. The environment depends directly on the operational profile. Since some of these factors are probabilistic in nature and operate over time, software reliability models are generally formulated in terms of random processes. The models are distinguished from each other, in general terms, by the nature of the variation of the random process with time.

A software reliability model specifies the general form of the dependence of the failure process on the factors mentioned above. We have assumed that it is, by definition, time based; this is not to say that non-time-based models may not provide useful insights, but the time-based formulation is the one that has proved useful in practice. The possibilities for different mathematical forms to describe the failure process are almost limitless.

As with any emerging discipline, software reliability has produced its share of models. Software reliability models are propounded to assess the reliability of software either from specified parameters, which are assumed to be known, or from software-error generated data. The past few years have seen the introduction of a number of different models, developed in response to the urgent need of software engineers, system engineers and managers to quantify the concept of software reliability. In these models the emphasis is on failures rather than on faults. Failures occur during the execution of the program, and hence the notion of time plays an important role. Reliability is the probability that the program will not fail during a certain period of time under stated conditions.

Normally we deal with calendar time. We may like to know the probability that a given system will not fail in a one-week period. But the models are generally based on execution time. The execution time is the time spent by the machine actually executing the program. Reliability models based on execution time yield better results than those based on calendar time, because they are not distorted by the periods in

which the program is idle, waiting for input, or being maintained. In many cases a posteriori translation of execution time to calendar time is possible, and the calendar time component presented later in this chapter shows how it is done. To emphasize this distinction, execution time will be denoted by τ and calendar time by t.

When the software fails, we try to locate and repair the fault that caused the failure. In particular, this situation arises during the testing phase of the life cycle. Most of the reliability models are applicable for system testing, when the individual modules have been integrated into one system.

During system testing, failure behaviour may not follow a constant pattern and may change over time, since the faults detected are subsequently repaired. A stochastic process whose probability distribution changes over time is called non-homogeneous. The variation in time between successive failures is described in terms of a function $\mu(\tau)$, which denotes the average number of failures up to time τ. Alternatively we may define:

$\lambda(\tau)$ = failure intensity function (average number of failures per unit of time at time τ)

The reliability of a program increases through fault correction, and hence the failure intensity decreases. This is the central qualitative statement that all the models in this chapter try to make quantitative.

7.3.1 Basic Execution Time Model

The model was developed by J. D. Musa in 1979 and is based on execution time. It is assumed that failures may occur according to a non-homogeneous Poisson process (NHPP). A Poisson process is a counting process in which the number of events in any interval depends only on the length of the interval, the events occur one at a time, and the events in disjoint intervals are independent. When the rate of the process is allowed to change with time, the process becomes non-homogeneous, and it is then described by an intensity function $\lambda(\tau)$ rather than by a single rate.

The process is observed during the execution of the program, and two quantities are recorded: the time τ at which each failure occurs and the cumulative number of failures observed up to that time. If $N(\tau)$ denotes the number of failures observed in the interval from 0 to τ, then the mean value function is $\mu(\tau) = E[N(\tau)]$, and the failure intensity is the derivative

$$\lambda(\tau) = d\mu(\tau)/d\tau$$

so that $\mu(\tau)$ is the integral of the intensity from 0 to τ. The initial condition is $\mu(0) = 0$, since no failures have been observed before execution begins.

The basic execution time model assumes that the failure intensity is proportional to the number of faults remaining in the program. Let V_0 be the total number of failures that would be experienced if execution continued for an infinite time, and let λ_0 be the initial failure intensity, that is, the intensity at the beginning of the observation. The assumption leads to the linear relationship

$$\lambda(\mu) = \lambda_0 (1 - \mu/V_0) \quad (7.1)$$

where μ is the number of failures already experienced. The relationship is shown in Fig. 7.13. The interpretation is simple: each failure removes one fault, the removal of a fault reduces the failure intensity by a constant amount, and the intensity falls to zero when all V_0 failures have been experienced.

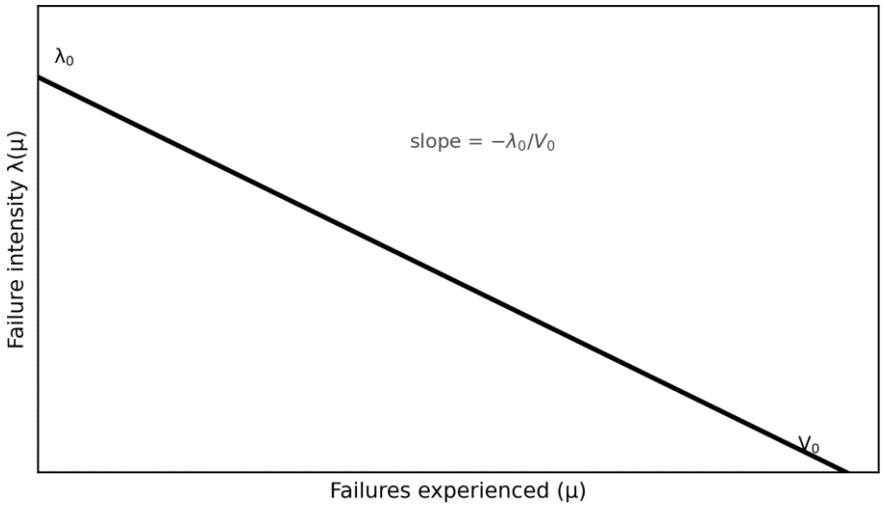


Figure 7.13. Failure intensity λ as a function of μ for the basic model.

Differentiating equation (7.1) with respect to μ gives the decrement of failure intensity per failure:

$$d\lambda/d\mu = -\lambda_0/V_0 \quad (7.2)$$

The decrement is a constant. It is the quantity that determines how much reliability improvement is obtained from each fault removal, and it is the first quantity that a project should estimate when it starts to use the model, because it can be obtained from the slope of the observed λ versus μ plot.

The model also gives the time behaviour. Substituting the differential equation for μ into equation (7.1) and solving with the initial condition $\mu(0) = 0$ gives

$$\mu(\tau) = V_0 [1 - \exp(-\lambda_0\tau/V_0)] \quad (7.3)$$

which is plotted in Fig. 7.14. The mean number of failures rises rapidly at first and then approaches the asymptote V_0. The failure intensity obtained by differentiating equation (7.3) is

$$\lambda(\tau) = \lambda_0 \exp(-\lambda_0\tau/V_0) \quad (7.4)$$

which is plotted in Fig. 7.15. The intensity decays exponentially towards zero. The two curves are two views of the same process: the cumulative curve saturates because the instantaneous rate is falling.

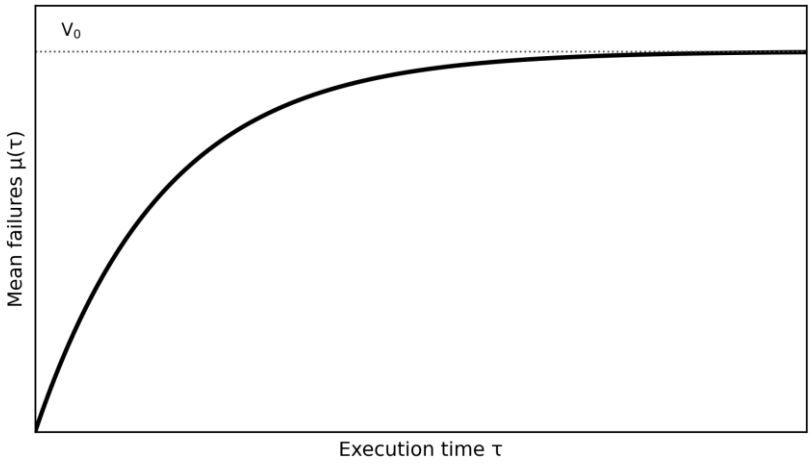


Figure 7.14. Relationship between τ and μ for the basic model.

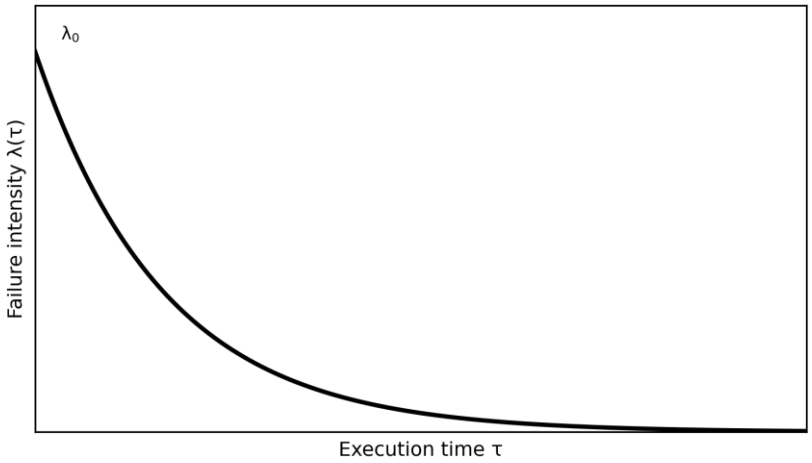


Figure 7.15. Failure intensity versus execution time for the basic model.

In practice the engineer is usually interested in a derived question rather than in the curve itself. Suppose that a failure intensity objective has been chosen for the product, and that some failures have already been corrected. How many additional failures must be experienced, and how much additional execution time is required, to reach the objective? Let λP be the present failure intensity and λF the objective, with λF < λP. The additional failures are obtained from equation (7.1):

$$\Delta\mu = (V_0/\lambda_0)(\lambda P - \lambda F) \quad (7.5)$$

and the additional execution time from equation (7.4):

$$\Delta\tau = (V_0/\lambda_0)\ln(\lambda P/\lambda F) \quad (7.6)$$

The two quantities are shown graphically in Fig. 7.16 and Fig. 7.17. Together they estimate the failure-correction workload, and they are the inputs to the calendar time component discussed in Section 7.3.3.

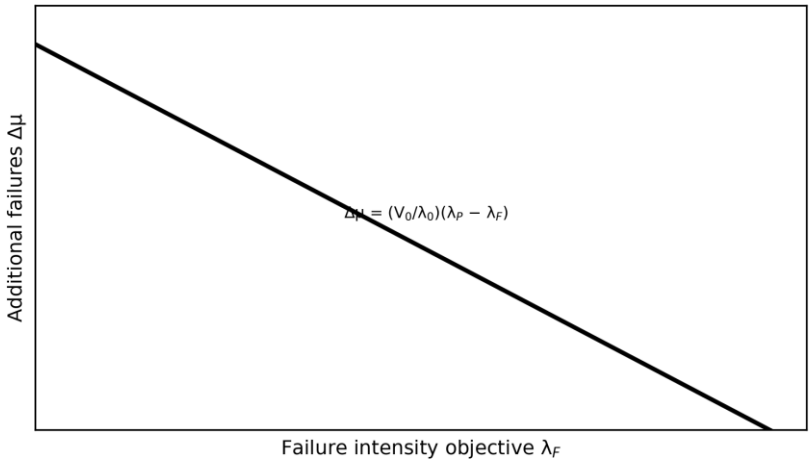


Figure 7.16. Additional failures required to reach the failure intensity objective.

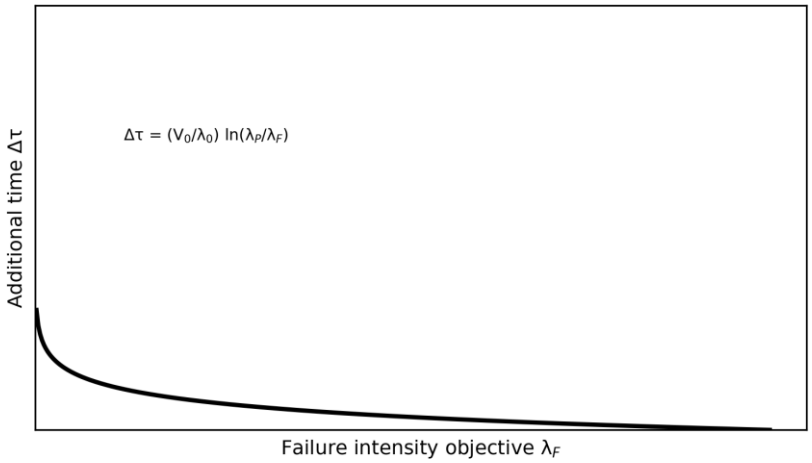


Figure 7.17. Additional time required to reach the objective.

Example 7.1. A program will experience 200 failures in infinite time. It has now experienced 100 failures. The initial failure intensity was 20 failures/CPU hr. (i) Determine the current failure intensity. (ii) Find the decrement of failure intensity per failure. (iii) Calculate the failures experienced and the failure intensity after 20 and after 100 CPU hr of execution. (iv) Compute the additional failures and the additional execution time required to reach the failure intensity objective of 5 failures/CPU hr. Use the basic execution time model.

Solution. Here $V_0 = 200$ failures, $\mu = 100$ failures, and $\lambda_0 = 20$ failures/CPU hr.

(i) Current failure intensity:

$\lambda(\mu) = \lambda_0(1 - \mu/V_0) = 20(1 - 100/200) = 20(0.5) = 10$ failures/CPU hr

(ii) Decrement of failure intensity per failure:

$d\lambda/d\mu = -\lambda_0/V_0 = -20/200 = -0.1$ per CPU hr

(iii)(a) Failures experienced and failure intensity after 20 CPU hr:

$\mu(20) = 200[1 - \exp(-20 \times 20/200)] = 200[1 - \exp(-2)] = 200(1 - 0.1353) = 173$ failures

$\lambda(20) = 20 \exp(-20 \times 20/200) = 20 \exp(-2) = 2.71$ failures/CPU hr

(iii)(b) After 100 CPU hr:

$\mu(100) = 200[1 - \exp(-10)] \approx 200$ failures

$\lambda(100) = 20 \exp(-10) = 0.000908$ failures/CPU hr

(iv) The present intensity is $\lambda P = 10$ and the objective is $\lambda F = 5$, so

$\Delta\mu = (V_0/\lambda_0)(\lambda P - \lambda F) = (200/20)(10 - 5) = 50$ failures

$\Delta\tau = (V_0/\lambda_0) \ln(\lambda P/\lambda F) = (200/20) \ln(10/5) = 6.93$ CPU hr

The example shows the characteristic behaviour of the model. The first 20 hours of execution remove 173 of the 200 eventual failures, and the intensity falls from 20 to 2.71 failures per CPU hour. The last 30 failures, however, occupy a disproportionate amount of time, because the intensity is already low when they are sought. This is the economic reason why testing is stopped at a stated objective rather than continued until no failures remain.

7.3.2 Logarithmic Poisson Execution Time Model

This model was also developed by Musa and his colleagues. The failure intensity is assumed to decay exponentially with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu) \quad (7.7)$$

where θ is the failure intensity decay parameter. The relationship is plotted in Fig. 7.18. Comparing equation (7.7) with equation (7.1), the difference is that the basic model makes the intensity fall linearly, whereas the logarithmic model makes it fall exponentially. The exponential form implies that the first fault removals produce a large reduction in intensity and that the later ones produce a progressively smaller reduction, which is closer to what is observed on many projects.

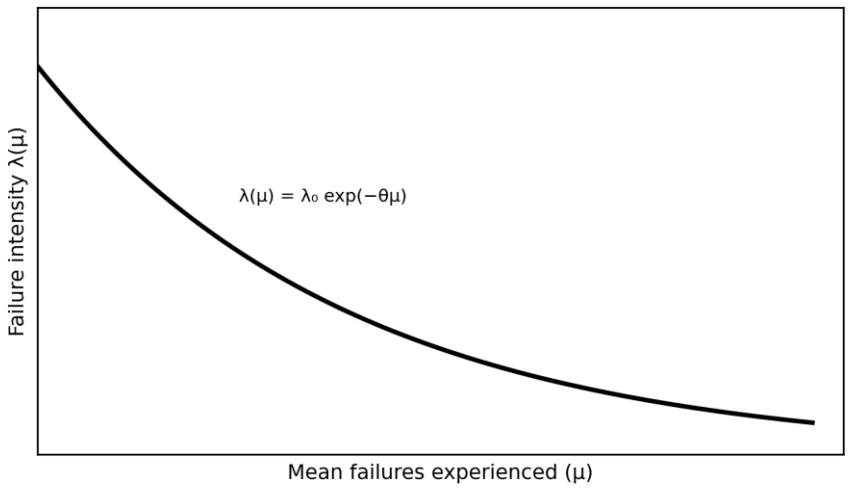


Figure 7.18. Relationship between μ and λ for the logarithmic model.

Inverting equation (7.7) gives the number of failures as a function of the intensity:

$\mu = (1/\theta) \ln(\lambda_0/\lambda)$

and the time behaviour is obtained by integrating the intensity. The results are

$$\lambda(\tau) = \lambda_0/(\lambda_0\theta\tau + 1) \quad (7.8)$$

and

$$\mu(\tau) = (1/\theta) \ln(\lambda_0\theta\tau + 1) \quad (7.9)$$

The mean value function grows logarithmically, as shown in Fig. 7.19, and it does not approach a finite asymptote: in this model an infinite number of failures would be experienced in infinite time. The absence of an asymptote is a real difference from the basic model, and it reflects the assumption that the program is never perfect.

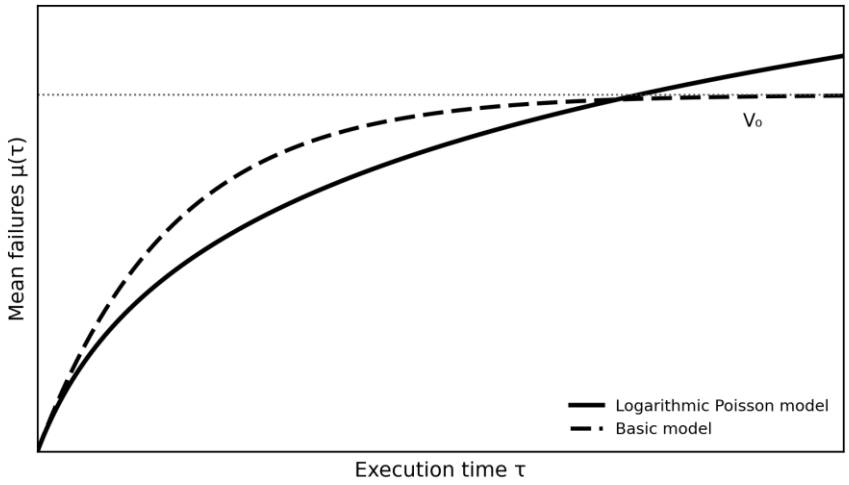


Figure 7.19. Relationship between μ and τ for the logarithmic model.

The derived quantities follow the same pattern as before. The additional failures required to move from the present intensity λP to the objective λF are

$$\Delta\mu = (1/\theta) \ln(\lambda P/\lambda F) \quad (7.10)$$

and the additional execution time is

$$\Delta\tau = (1/\theta)(1/\lambda F - 1/\lambda P) \quad (7.11)$$

Example 7.2. Assume that the initial failure intensity is 20 failures/CPU hr. The failure intensity decay parameter is 0.02 per failure. We have experienced 100 failures up to this time. (i) Determine the current failure intensity. (ii) Calculate the decrement of failure intensity per failure. (iii) Find the failures experienced and the failure intensity after 20 and after 100 CPU hr of execution. (iv) Compute the additional failures and the additional execution time required to reach the failure intensity objective of 2 failures/CPU hr. Use the logarithmic Poisson execution time model.

Solution. Here $\lambda_0 = 20$ failures/CPU hr, $\mu = 100$ failures and $\theta = 0.02$ per failure.

(i) Current failure intensity:

$\lambda(\mu) = \lambda_0 \exp(-\theta\mu) = 20 \exp(-0.02 \times 100) = 2.7$ failures/CPU hr

(ii) Decrement of failure intensity per failure:

$d\lambda/d\mu = -\theta\lambda = -0.02 \times 2.7 = -0.054$ per CPU hr

(iii)(a) Failures experienced and failure intensity after 20 CPU hr:

$\mu(20) = (1/0.02) \ln(20 \times 0.02 \times 20 + 1) = 109$ failures

$\lambda(20) = 20/(20 \times 0.02 \times 20 + 1) = 2.22$ failures/CPU hr

(iii)(b) After 100 CPU hr:

$\mu(100) = (1/0.02) \ln(20 \times 0.02 \times 100 + 1) = 186$ failures

$\lambda(100) = 20/(20 \times 0.02 \times 100 + 1) = 0.4878$ failures/CPU hr

(iv) With $\lambda P = 2.7$ and $\lambda F = 2$,

$\Delta\mu = (1/0.02) \ln(2.7/2) = 15$ failures

$\Delta\tau = (1/0.02)[1/2 - 1/2.7] = 6.5$ CPU hr

Example 7.3. The following parameters are given for the basic and the logarithmic Poisson execution time models: for the basic model $\lambda_0 = 10$ failures/CPU hr and $V_0 = 100$ failures; for the logarithmic model $\lambda_0 = 30$ failures/CPU hr and $\theta = 0.025$ per failure. (a) Determine the additional failures and the additional execution time required to reach the failure intensity objective of 5 failures/CPU hr for both models. (b) Repeat this for an objective of 0.5 failures/CPU hr, assuming that we start with the initial failure intensity only.

Solution. (a)(i) Basic execution time model, with $\lambda P = \lambda_0 = 10$ and $\lambda F = 5$:

$\Delta\mu = (V_0/\lambda_0)(\lambda P - \lambda F) = (100/10)(10 - 5) = 50$ failures

$\Delta\tau = (V_0/\lambda_0) \ln(\lambda P/\lambda F) = (100/10) \ln(10/5) = 6.93$ CPU hr

(a)(ii) Logarithmic execution time model, with $\lambda P = \lambda_0 = 30$ and $\lambda F = 5$:

$\Delta\mu = (1/\theta) \ln(\lambda P/\lambda F) = (1/0.025) \ln(30/5) = 71.67$ failures

$\Delta\tau = (1/\theta)(1/\lambda F - 1/\lambda P) = (1/0.025)(1/5 - 1/30) = 6.66$ CPU hr

(b)(i) Basic model with $\lambda F = 0.5$:

$\Delta\mu = (100/10)(10 - 0.5) = 95$ failures

$\Delta\tau = (100/10) \ln(10/0.5) = 30$ CPU hr

(b)(ii) Logarithmic model with $\lambda F = 0.5$:

$\Delta\mu = (1/0.025) \ln(30/0.5) = 163.8$ failures

$\Delta\tau = (1/0.025)(1/0.5 - 1/30) = 78.66$ CPU hr

The comparison is instructive. At the objective of 5 failures per CPU hour the logarithmic model predicts substantially more failures than the basic model, 71.67 against 50, but slightly less execution time, 6.66 against 6.93 CPU hr. When the objective is tightened to 0.5 failures per CPU hour the gap in both quantities widens: the logarithmic model now predicts about 164 failures against 95, and about 78.66 CPU hr against 30. The logarithmic model therefore calculates more failures in the same duration of execution time initially, and as the failure intensity objective becomes smaller, the additional failures and additional time required become substantially larger for the logarithmic model than for the basic model.

That difference is not a mathematical curiosity. It expresses a different belief about the program. The basic model believes that the program contains a fixed, finite number of faults and that testing removes them until the program is effectively perfect. The logarithmic model believes that the difficulty of finding the next fault increases as the obvious faults are removed, so that the same amount of testing buys less and less improvement. Choosing between the models is therefore a matter of evidence about the project, and the evidence is obtained by fitting both models to the failure data collected during test.

7.3.3 Calendar Time Component

The calendar time component relates execution time and calendar time by determining the calendar time to execution time ratio at any given point in time. The ratio is based on the constraints that are involved in applying resources to a project. To obtain calendar time, one integrates this ratio with respect to execution time. The calendar time component is of greatest significance during phases where the software is being tested and repaired, because during this period one can predict the dates at which various failure intensity objectives will be met. The calendar time component exists during periods in which repair is not occurring and failure intensity is constant; however, it reduces in that case to a constant ratio between calendar time and execution time.

In test, the rate of testing at any time is constrained by the failure identification or test team personnel, the failure correction or debugging personnel, or the computer time available. The quantities of these resources available to a project are usually more or less established in its early stages. Increases are generally not feasible during the system test phase because of the long lead times required for training and computer procurement. At any given value of execution time, one of these resources will be limiting. The limiting resource determines the rate at which execution time can be spent per unit of calendar time. A test phase may consist of from one to three periods, each characterized by a different limiting resource.

The following is a common scenario. At the start of testing one identifies a large number of failures separated by short time intervals. Testing must be stopped from time to time to let the people who are fixing the faults keep up with the load. As testing progresses, the intervals between failures become longer and longer. The time of the failure correction personnel is no longer completely filled with failure correction work. The test team becomes the bottleneck. The effort required to run tests and analyse the results is occupying all their time. That paces the amount of testing done each day. Finally, at even longer intervals, the capacity of the computing facilities becomes limiting. This resource then determines how much testing is accomplished.

The calendar time component is therefore based on a debugging process model. This model takes into account the resources used in operating the program for a given execution time and processing an associated quantity of failures, the resource quantities available, and the degree to which a resource can be utilized, due to bottlenecks, during the period in which it is limiting.

Resource usage is linearly proportional to execution time and to the mean number of failures experienced. Let xr be the usage of resource r. Then

$$x_r = \theta_r \tau + \mu_r \mu \quad (7.12)$$

Note that θ_r is the resource usage per CPU hr. It is nonzero for failure identification personnel, θ_I, and for computer time, θ_C. The quantity μ_r is the resource usage per failure. Be careful not to confuse it with the mean failures experienced μ. It is nonzero for failure identification personnel, μ_I, failure correction personnel, μ_F, and computer time, μ_C.

Table 7.9: Calendar time component resources and parameters

Resource	Usage per CPU hr	Usage per failure	Quantity available	Utilization
Failure identification personnel	θ_I	μ_I	P_I	1
Failure correction personnel	0	μ_F	P_F	ρ_F
Computer time	θ_C	μ_C	P_C	ρ_C

For failure correction, unlike identification, the resources required depend only on the mean failures experienced. However, computer time is used in both identification and correction, so computer time used usually depends on both the amount of execution time and the number of failures.

Computer time required per unit execution time is normally greater than 1. In addition to the execution time for the program under test, additional time is required for the execution of support programs such as test drivers, recording routines and data reduction packages.

Consider the change in resource usage per unit of execution time. It is obtained by differentiating equation (7.12) with respect to execution time:

$$dx_r/d\tau = \theta_r + \mu_r \lambda \quad (7.13)$$

Since the failure intensity decreases with testing, the effort used per hour of execution time tends to decrease with testing. It approaches the execution time coefficient of resource usage asymptotically as execution time increases.

Resource quantities and utilizations are assumed to be constant for the period over which the model is being applied. This is a reasonable assumption, as increases are usually not feasible. The instantaneous ratio of calendar time to execution time is obtained by dividing the resource usage rate of the limiting resource by the constant quantity of resources available that can be utilized. Let t be calendar time. Then

$$dt/d\tau = (1/P_r \rho_r)\, dx_r/d\tau \quad (7.14)$$

The quantity P_r represents the resource available, and ρ_r is the utilization. The above ratio must be computed separately for each resource-limited period, because x_r is a function of τ. We thus have a relationship between t and τ in each resource-limited period. The form of the instantaneous calendar time to execution time ratio for any given limiting resource and either model is readily obtained from equations (7.13) and (7.14) as

$$dt/d\tau = (\theta_r + \mu_r \lambda)/P_r \rho_r \quad (7.15)$$

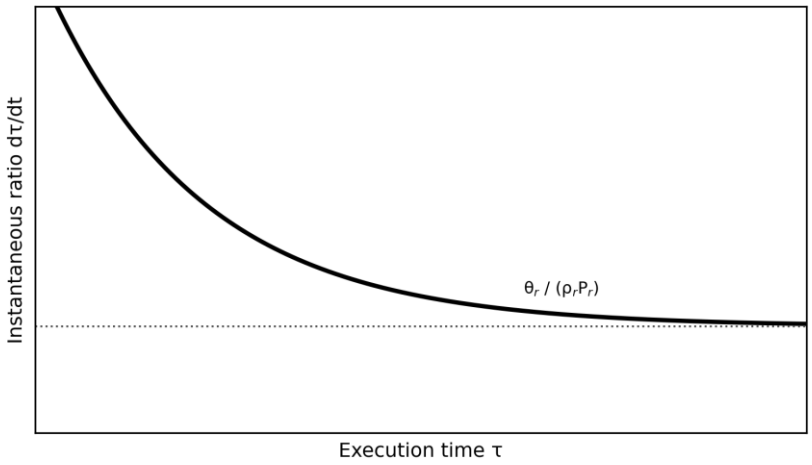


Figure 7.20. Instantaneous calendar time to execution time ratio for a given limiting resource.

The shape of this curve parallels that of the failure intensity. The curve approaches an asymptote of θr/(Pr ρr). Note that the asymptote is zero for the failure correction personnel, the maximum of the ratios for the three limiting resources. At any given time, the resource with the maximum ratio determines the rate at which calendar time is expanded; this is illustrated in Fig. 7.21. The curve for a resource is plotted as a solid curve when that resource is limiting and as a thin curve when it is not. Note the transaction points FI and IC. Here the calendar time to execution time ratios of two resources are equal and the limiting resource changes. The point FC is a potential but not a true transition point, because neither resource F nor resource C is limiting near this point.

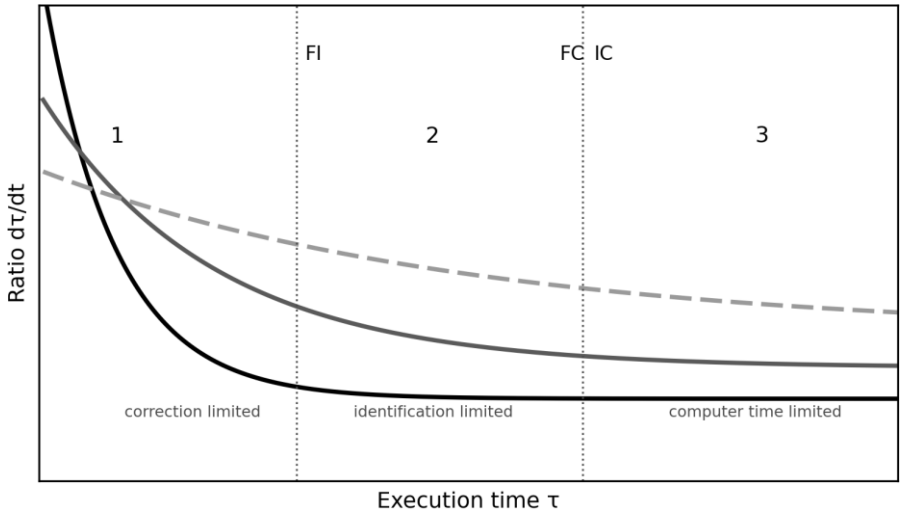


Figure 7.21. Calendar time to execution time ratio for different limiting resources.

The calendar time component allows the engineer to estimate the calendar time in days required to meet the failure intensity objective. The value of this interval is particularly useful to software managers and engineers. One may determine it from the additional execution time and additional number of failures needed to meet the objective that were found for the execution time component. Second, one now knows the date on which the failure intensity objective will be achieved. This is a simple variant of the first

quantity that takes account of weekends and holidays. It is a useful quantity because it speaks in the units that managers understand.

Example 7.4. A team runs test cases for 10 CPU hr and identifies 25 failures. The effort required per hour of execution time is 5 person hr. Each failure requires 2 hr on average to verify and determine its nature. Calculate the failure identification effort required.

Solution. Using $x_r = \theta_r \tau + \mu_r \mu$ with $\theta_I = 5$ person hr/hr, $\tau = 10$ CPU hr, $\mu_I = 2$ hr/failure, $\mu = 25$ failures:

$$X_I = 5(10) + 2(25) = 50 + 50 = 100 \text{ person hr}$$

Example 7.5. The initial failure intensity λ_0 for a given software is 20 failures/CPU hr. The failure intensity objective λ_F of 1 failure/CPU hr is to be achieved. The resource usage parameters are 2 person hr per hour and 1 person hr per failure for failure identification, 5 person hr per failure for failure correction, and 1.5 CPU hr per hour and 1 CPU hr per failure for computer time. (a) What resources must be expended to achieve the reliability improvement? Use the logarithmic Poisson model with $\theta = 0.025$ per failure. (b) If the failure intensity objective is cut to half, what is the effect on the requirement of resources?

Solution. (a) From equations (7.10) and (7.11),

$$\Delta\mu = (1/0.025) \ln(20/1) = 119 \text{ failures}$$

$$\Delta\tau = (1/0.025)(1/1 - 1/20) = 38 \text{ CPU hr}$$

Hence

$$X_I = \mu_I \Delta\mu + \theta_I \Delta\tau = 1(119) + 2(38) = 195 \text{ person hr}$$

$$X_F = \mu_F \Delta\mu = 5(119) = 595 \text{ person hr}$$

$$X_C = \mu_C \Delta\mu + \theta_C \Delta\tau = 1(119) + 1.5(38) = 176 \text{ CPU hr}$$

(b) With $\lambda_F = 0.5$ failures/CPU hr,

$$\Delta\mu = (1/0.025) \ln(20/0.5) \approx 148 \text{ failures}$$

$$\Delta\tau = (1/0.025)(1/0.5 - 1/20) = 78 \text{ CPU hr}$$

$$X_I = 1(148) + 2(78) = 304 \text{ person hr}$$

$$X_F = 5(148) = 740 \text{ person hr}$$

$$X_C = 1(148) + 1.5(78) = 265 \text{ CPU hr}$$

Hence, if we cut the failure intensity objective to half, resource requirements are not doubled but they are somewhat less. Note that $\Delta\tau$ is approximately doubled but increases logarithmically. Thus, the resources increase will be between a logarithmic increase and a linear increase for changes in failure intensity objective.

Example 7.6. A program is expected to have 500 faults. It is also assumed that one fault may lead to one failure only. The initial failure intensity was 2 failures/CPU hr. The program was to be released with a failure intensity objective of 5 failures/100 CPU hr. Calculate the number of failures experienced before release.

Solution. The number of failures experienced during testing is calculated by

$\Delta\mu = (V_o/\lambda_o)(\lambda P - \lambda F)$

Here $V_o = 500$, because one fault leads to one failure; $\lambda_o = 2$ failures/CPU hr; and $\lambda F = 5$ failures/100 CPU hr = 0.05 failures/CPU hr, so

$\Delta\mu = (500/2)(2 - 0.05) = 487$ failures

Hence 13 faults are expected to remain at the release instant of the software.

7.3.4 The Jelinski-Moranda Model

The Jelinski-Moranda model is the earliest and probably the best-known reliability model. It proposes a failure intensity function of the form

$$\lambda(t) = \varphi(N - i + 1) \quad (7.16)$$

where φ is the constant of proportionality, N is the total number of errors present, and i is the number of errors found by time interval ti.

The model assumes that all failures have the same failure rate. It means that the failure rate is a step function and that there will be an improvement in reliability after fixing a fault, so that every failure contributes equally to the overall reliability. Here, the failure intensity is directly proportional to the number of remaining errors in a program. The relation between time and failure intensity is shown in Fig. 7.22.

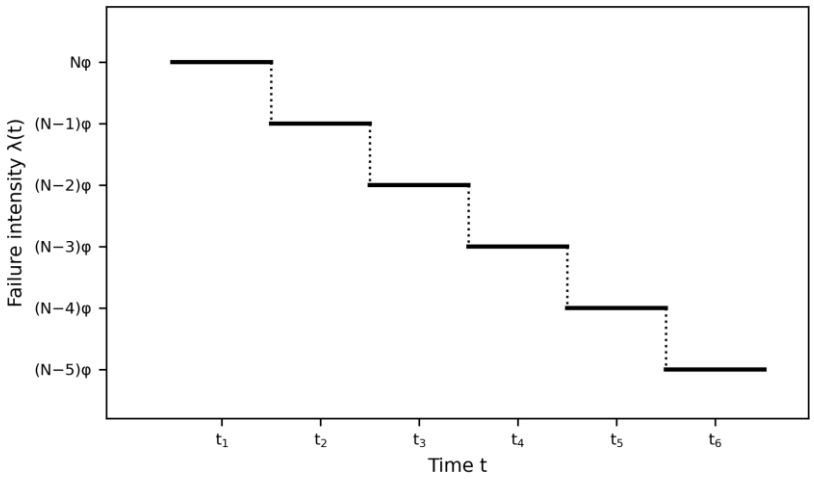


Figure 7.22. Relation between t and λ for the Jelinski-Moranda model.

The time intervals $t_1, t_2, ..., t_k$ may vary in duration depending upon the occurrence of a failure. Between the $(i - 1)$th and the ith failure, the failure intensity function is $(N - i + 1)\varphi$. The model is therefore a step function: the intensity is constant between failures and drops by φ at each failure.

Example 7.7. There are 100 errors estimated to be present in a program. We have experienced 60 errors. Use the Jelinski-Moranda model to calculate the failure intensity with a given value of $\varphi = 0.03$. What will be the failure intensity after the experience of 80 errors?

Solution. With N = 100, i = 60 and $\varphi = 0.03$,

$\lambda(t) = \varphi(N - i + 1) = 0.03(100 - 60 + 1) = 1.23$ failures/CPU hr

After 80 failures,

$\lambda(t) = 0.03(100 - 80 + 1) = 0.63$ failures/CPU hr

Hence there is a continuous decrease in the failure intensity as the number of failures experienced increases.

7.3.5 The Bug Seeding Model

The so-called bug seeding model is an outgrowth of a technique used to estimate the number of animals in a wild life population or fish in a pond. The technique is best illustrated by discussing the estimation of the number of a specific species of fish, say bass, N, in a small pond which contains no other type of fish. We begin by procuring a suitable number of bass, Nt, from a fish hatchery and tag each one with a means of identification which will remain reliably attached for the length of the measurement period. The Nt tagged fish are then added to the N original fish and allowed to mix and disperse. After an appropriate number of days a sample is fished from the lake, by hook or net, and separated into nt tagged bass and n untagged bass. If we assume that there was no difference in the dispersion or ease of catching of the tagged and untagged fish, then we can set up the following equation, which equals the proportions of tagged fish in the fished sample to the original fraction seeded:

$$Nt/(N + Nt) = nt/(n + nt) \quad (7.17)$$

We may solve the above equation for N in terms of the known quantities Nt, n and nt, yielding

$$\hat{N} = (n/nt)\, Nt \quad (7.18)$$

By direct analogy we may consider N to be the unknown number of bugs in the program at the start of debugging and Ns = Nt to be the number of seeded bugs, unknown to the debugger. After γ months of debugging, the bugs which have been removed are examined by someone other than the debugger, who has a list of the seeded bugs. This tester classifies them as ns, which come from the seeded group, and n, which were not seeded. Direct substitution yields

$$\hat{N} = (n/ns)\, Ns \quad (7.19)$$

The possibility of seeding bugs in a program and of using this technique to measure the initial bug content was first suggested by Mills. The results of the early experiments in this area were inconclusive for two reasons. First of all, it was difficult to make up realistic bugs. Consider the analogous fish problem of matching ages of the bass, not to mention the realistic situation where one has, say, three types of fish — bass, perch and pickerel. Now one must also match the unknown ratios in seeding the tagged fish unless all types of fish are equally easy to catch. Also, are hatchery-bred fish as easy to catch as pond-bred fish? The second problem with the early seeding experiments was that the seeding was used to evaluate the effectiveness of coder reading as opposed to machine testing, a means of debugging programs. Thus, the results of the two experiments somewhat clouded a crisp evaluation of either.

A different approach was suggested by Hyman, which circumvented the problem of seeding bugs. He proposed that one employ two (or more) independent debuggers working on the same program initially. Suppose that it is estimated that debugging will take 4 months and that debugger number 1 is assigned to

the program for 4 months, or the duration of the job. Debugger number 2 is assigned to the job for only one or two months at the beginning of the project. The two debuggers work independently, and after a few weeks the results of their efforts are evaluated by a third analyst, who estimates the number of program bugs N.

The estimates are repeated every few weeks, and when the third analyst is satisfied that the value N is sufficiently well estimated, the results of debugger number 2's work are given to debugger number 1 and debugger number 2 is reassigned. Now, reasonable estimates of the total number of bugs in the program, and knowing the number of bugs already removed, by subtraction we obtain the number of remaining bugs. In addition, only a portion of debugger number 2's findings duplicate debugger number 1's. Thus, debugger number 1 is able to rapidly incorporate much of debugger number 2's work, thereby producing almost a step change in the number of bugs found. In most cases the benefits should far outweigh the costs — one or two extra man-months of debugging time; the cost may be minor, zero, or negative if debugger number 2 really finds many independent errors which shorten debugger number 1's efforts.

In order to develop the two-debugger estimation procedure, we begin with the following notation: γ is the development time in months, and the interval from 0 to γ_1 is the measurement period; B_0 is the number of bugs in the program at $\gamma = 0$; B_1 is the number of bugs found by debugger number 1 up to time γ_1; b_0 is the number of bugs which debugger number 2 finds up to time γ_1 which are common, that is, in set B_1; and b_1 is the number of bugs which programmer number 2 finds up to time γ_1 which are independent, that is, not in set B_1. Then $B_2 = b_0 + b_1$ is the number of bugs found in the program by debugger number 2 up to time γ_1.

If we really believe that the bugs we would seed in a bug seeding experiment are identical to the indigenous bugs, then instead of seeding we could merely locate a certain number of bugs and tag them, that is, identify them. Of course, the problem here is that much work must transpire in order to identify a group of bugs. Since it should not matter which set of bugs we choose as the tagged set, as long as they are representative, we should be able to treat the identified set as if they were a tagged set of bugs. In order to proceed, we must make some fundamental assumptions, which we state as hypotheses.

Bug characteristics unchanged as debugging proceeds. When a large program is debugged, the bugs found during the first several weeks or months are representative of the total bug population.

Independent debugging results in similar programs. When two independent debuggers work on a large program, the evolution of the program is such that the differences between the two versions are small enough that they can be neglected.

Common bugs are representative. When two independent debuggers work on a large program, the bugs which they find in common are representative of the total population.

Assuming that the above hypotheses are valid, we can treat the quantity B_1 as if it were the selected group Ns. Similarly, b_0 becomes ns and B_2 becomes n. Substituting these quantities, solving for the original number of bugs $B_0 = N$, yields

$$\hat{B}_0 = B_1 B_2 / b_0 \quad (7.20)$$

7.3.6 Choosing Among the Models

The models presented in this section are not competing descriptions of the same truth; they are different simplifying assumptions about how faults are distributed and how testing removes them. The differences are summarised in Table 7.10.

Table 7.10: Comparison of the reliability models

Model	Basic assumption	Behaviour	Main use
Basic execution time	Failure intensity falls linearly with failures experienced; finite total failures V_0	Exponential decay of intensity; mean failures saturate at V_0	Projects whose fault population is approximately fixed and whose early testing is representative
Logarithmic Poisson	Intensity falls exponentially with failures experienced; no finite asymptote	Logarithmic growth of mean failures; slower improvement per failure as testing proceeds	Projects where the easily found faults are removed first and the remaining ones are progressively harder
Jelinski-Moranda	Each fault contributes equally; intensity proportional to remaining faults	Step function; equal decrement φ at each failure	Simple teaching model and early estimation from failure intervals
Bug seeding	The proportion of seeded bugs found estimates the total population	Single estimate of the initial bug count, refined by repetition	Estimating the total defect content when independent debuggers are available
Calendar time	Execution time is converted to calendar time through the limiting resource	Piecewise ratio; the resource with the largest ratio paces the project	Planning test completion dates and resource levels

Two practical warnings follow from the table. The first is that a model should be selected after the failure data have been inspected, not before. If the observed intensity falls steeply and then flattens, the logarithmic model is the better description; if it falls steadily towards zero, the basic model is adequate. The second is that every model is an extrapolation. It describes the future behaviour of the program under the assumption that the testing process and the operational profile remain unchanged, and that assumption is precisely the one that fails when a project changes its test strategy, its team or its target environment. The value of the models is that they make the assumption explicit and allow the engineer to ask what would happen if it held.

7.3.7 Reliability Growth Models

A reliability growth model is a mathematical model of how software reliability improves as errors are detected and repaired. A reliability growth model can be used to predict when, or if at all, a particular level of reliability is likely to be attained. Thus, reliability growth modelling can be used to determine when to stop testing to attain a given reliability level. Although several different reliability growth models have been proposed, two very simple ones illustrate the range of assumptions that such models make.

Littlewood and Verall's model. This model allows for negative reliability growth to reflect the fact that when a repair is carried out, it may introduce additional errors. It also models the fact that, as errors are repaired, the average improvement in reliability per repair decreases. It treats an error's contribution to reliability improvement as an independent random variable having a Gamma distribution. This distribution models the fact that error corrections with large contributions to reliability growth are removed first. This represents diminishing return as testing continues.

The model is useful because it captures two observations that the simpler models ignore. The first is that a repair is not guaranteed to improve the program: a badly made change can make the next failure more

likely, and the model admits this possibility by allowing the growth to be negative. The second is that the faults which are easy to find are not necessarily the faults whose correction produces the largest improvement. The Gamma assumption reproduces the practical experience that the largest improvements come early and that later repairs buy less.

The step function model. The simplest reliability growth model is a step function model, in which it is assumed that the reliability increases by a constant increment each time an error is detected and repaired. However, this simple model, which implicitly assumes that all errors contribute equally to reliability growth, is highly unrealistic, since we already know that corrections of different errors contribute differently to reliability growth.

There are more complex reliability growth models that give more accurate approximations to reliability growth, but they are beyond the scope of this chapter. The practical lesson is that the choice of a growth model determines the shape of the prediction: the step function predicts a staircase with equal steps, the Jelinski–Moranda model predicts a staircase whose steps have equal height but unequal duration, and the Littlewood–Verall model predicts a curve whose slope changes as testing proceeds.

Table 7.11: Assumptions of the reliability growth models

Model	Assumption about each repair	Shape of the predicted growth	Main limitation
Step function	Every repair adds the same increment of reliability	Staircase with equal steps	Unrealistic: errors differ in their contribution
Jelinski–Moranda	Every remaining fault has the same failure rate	Staircase dropping by a constant amount at each failure	Assumes all faults are equally likely to be exposed
Littlewood–Verall	The contribution of each repair is a random variable with a Gamma distribution	Curve with decreasing slope, possibly negative growth	Requires more data to estimate the distribution
Basic execution time	Failure intensity is proportional to the remaining faults	Exponential decay towards zero at V_0	Assumes a finite, fixed fault population
Logarithmic Poisson	Intensity decays exponentially with failures experienced	Logarithmic growth without an asymptote	Assumes the program is never perfect

7.3.8 Reliability Assessment

In general, many properties of engineering artefacts, such as reliability, are measured and verified in this way. For instance, the reliability of an electrical appliance may be measured in terms of its probability of failure within a given time. This measure is helpful whenever we cannot guarantee the absence of failures absolutely. Software reliability is the probability of the failure-free operation of a computer program for a specified time in a specified environment.

The process of measuring the reliability of a system involves four stages.

1. Existing systems of the same type are studied to establish an operational profile. An operational profile identifies different classes of system inputs and the probability of these inputs in normal use.
2. A set of test data is constructed, sometimes with the help of test-data generators, that reflects the operational profile.
3. The system is tested with these data and the number of failures is observed. The times of these failures are also logged; the time units chosen should be appropriate for the reliability metric used.

4. After a statistically significant number of failures have been observed, the software reliability can then be computed, and the appropriate reliability metric value can be worked out.

This approach to reliability measurement is not easy to apply in practice, and the principal difficulties arise from three sources. The first is operational profile uncertainty: the operational profiles may not be an accurate reflection of the real use of the system. The second is the high cost of test-data generation: defining a large amount of test data takes a long time if it is not possible to generate this data automatically. The third is statistical uncertainty when a high reliability is specified: it is important to generate a statistically significant number of failures to allow accurate reliability measurements.

These difficulties explain why reliability assessment is usually a combination of measurement and judgement rather than a purely computational exercise. A profile that is wrong will produce a reliability estimate that is precise but false; test data that is too small will produce an estimate with a wide confidence interval; and a target that is very high will require a number of failures that the schedule cannot afford to observe. The engineer who understands these limits is in a better position than the engineer who trusts a single number produced by a model.

7.3.9 Statistical Testing and the Operational Profile

The models in this chapter take failure data as their input, and the quality of a model's output can never exceed the quality of the data. The discipline that produces the data is statistical testing, and the definition of the environment within which the data are collected is the operational profile, discussed earlier in Section 7.1.4.

Statistical testing means that the test cases are selected in proportion to the way the software will be used. The operational profile identifies the classes of system inputs and the probability of each class in normal use; the test data set is constructed so that its distribution matches that profile; the system is then tested with these data and the number of failures is observed. The failure times are logged, and the time unit chosen must match the reliability metric that will be computed: CPU hours for an execution time model, transactions for a rate of occurrence of failure, and service requests for a probability of failure on demand.

The advantage of the approach is that the reliability estimate refers to the environment that the user actually experiences. A test suite that exercises every branch uniformly gives a measure of coverage, but it does not give a measure of the probability of failure in operation, because the branches are not exercised in operation with uniform frequency. A statistical test gives the second measure, and it is the one that the reliability models consume.

The disadvantage is that statistical testing concentrates the effort on the parts of the system that are most likely to be used. This is a virtue for the user, who experiences fewer failures, and a danger for the engineer, who may conclude that the software is more reliable than it is, because the rare inputs that trigger the severe failures are by construction rare in the test set. For this reason a statistical test suite is usually supplemented by a deliberate search for the unlikely inputs: the rare combination, the boundary value and the invalid data that the profile assigns a very small probability but whose consequences are not small at all.

Creating those unlikely inputs with a test case generator is difficult. A generator can sample from the profile, but it cannot invent the input that a malicious or unlucky user will produce. The practical compromise is to generate the bulk of the test data from the profile and to add, by hand, a small number

of cases for each hazardous class of input. The reliability estimate is then computed from the statistical part, while the hazardous cases are treated as a separate safety argument.

There is a second, subtler difficulty. A statistical test measures the software that is tested, in the environment in which it is tested, with the profile that was assumed. If the profile is wrong, the estimate is precise but false, and the precision makes the error harder to detect. This is why the operational profile is itself a deliverable that must be reviewed: it is a model of the user, and a model of the user can be wrong in the same way as a model of the software.

7.3.10 Uses of Reliability Studies

Pressures have been increasing for achieving a more finely tuned balance among product and process characteristics, including reliability. Trade-offs among product components with respect to reliability are also becoming increasingly important. Thus an important use of software reliability measurement is in system engineering. There are at least four other ways in which software reliability measures can be of great value to the software engineer, the manager or the user.

First, you can use software reliability measures to evaluate software engineering technology quantitatively. New techniques are continually being proposed for the process of developing software, but unfortunately they have been exposed to little quantitative evaluation. Because of the inability to distinguish between good and bad, new technology is often led into a general resistance to change that is counterproductive. Reliability measures offer the promise of establishing at least one criterion for evaluating technology; for example, you might run experiments to determine the decrease in failure intensity at the start of system test resulting from design reviews. A quantitative evaluation such as this makes the benefits of good software engineering technology highly visible.

Second, software reliability measures offer you the possibility of evaluating development status during the test phases of a project. Methods such as the intuition of designers or of the test team, the percentage of tests completed, and the successful execution of critical functional tests have been used to evaluate testing progress. None of these has been really satisfactory, and none has been quite as sound a means of determining status as a well-defined reliability measure, such as failure intensity. Reliability generally increases with the amount of testing, so reliability can be closely linked with project schedules. Moreover, the cost of testing is highly correlated with failure intensity improvement. Since the process attributes that a manager must control are schedule and cost, reliability is thus intimately tied to project management.

Third, one can use software reliability measures to monitor the operational performance of software and to control new features added and design changes made to the software. The reliability of software usually decreases as a result of such changes. A reliability objective can be used to determine when, and perhaps how large, a change will be allowed. The objective would be based on the operational profile of the users, and the measure would be taken after the change has been in service for a stated period.

Fourth, a quantitative understanding of software quality and the various factors influencing it and affected by it enriches the software product and the software development process. A manager who can say that the failure intensity of the product fell from one release to the next, and can attribute the fall to specific changes in the process, is in a much stronger position than a manager who can only say that the product feels better. The measurement turns an opinion into evidence, and evidence is what allows an organization to repeat a success instead of remembering it.

The four uses share a common logic. In each case the reliability measure replaces a judgement that would otherwise be subjective, and in each case the value of the measure depends on the same three conditions: the environment must be stated, the time period must be stated, and the operational profile must be known.

7.3.11 Verification, Validation and Reliability

The reliability models predict what will happen; they do not by themselves make it happen. The activities that make it happen are verification and validation, and their relationship to reliability is worth stating explicitly.

Verification asks whether the system conforms to its specification; validation asks whether the system does what the user actually needs. A system can be verified and still be unreliable, because the specification may be incomplete, and it can be validated on the cases that were tried and still be unreliable, because the cases that were not tried may be the ones that fail. Reliability, in the sense of this chapter, is a statement about the probability of failure over time in a stated environment, and neither verification nor validation alone can establish it.

Static analysis is an approach to verification and validation that examines the source code, or another representation of the system, looking for errors and anomalies. Its advantage is coverage: it allows all parts of a program to be checked, not just those parts that are exercised by the tests. A fault in a branch that no test reaches is invisible to testing and visible to analysis, which is why the two techniques are complementary rather than alternative.

Model checking is a formal approach to static analysis that exhaustively checks all the states of a system for potential errors. It is powerful where the state space is finite and the property to be checked can be stated precisely, and it is expensive where neither condition holds. For a reliability argument, model checking is most useful for the properties whose failure would be catastrophic and whose occurrence is too rare to be observed in testing.

Statistical testing, discussed above, estimates reliability by testing the system with a data set that reflects the operational profile. It is the only one of the three techniques that produces a quantitative estimate of the probability of failure in operation, and that is why the models of this chapter depend on it.

The three techniques answer three different questions. Static analysis answers whether there is a fault; model checking answers whether a stated property can ever be violated; statistical testing answers how often the system is likely to fail in use. A reliability programme that relies on only one of them is answering only one of the three questions, and the answer it gives is likely to be misinterpreted as an answer to the other two.

7.4 Capability Maturity Model

The capability maturity model (CMM) is not a software life cycle model. Instead, it is a strategy for improving the software process, irrespective of the actual life cycle model used. The CMM was developed by the Software Engineering Institute (SEI) of Carnegie-Mellon University in 1986. It is used to judge the maturity of the software processes of an organization and to identify the key practices that are required to increase the maturity of these processes. The CMM is organized into five maturity levels, as shown in Fig. 7.23.

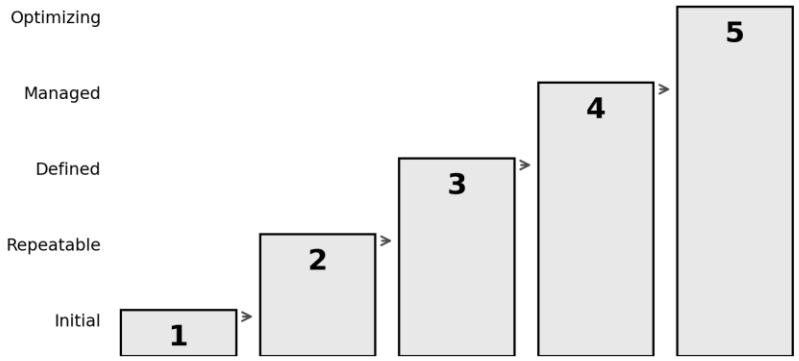


Figure 7.23. Maturity levels of CMM.

The model rests on a simple but demanding idea: the quality of a software product is largely determined by the quality of the process used to develop and maintain it, and a process can be improved in a sequence of measurable stages. Each stage is a level of maturity, and each level is defined by the set of process areas that the organization must master before it can move to the next level. The levels are ordered, so an organization cannot skip a level: the practices of level n+1 assume that the practices of level n are already institutionalised.

7.4.1 Maturity Levels

1. Initial (Maturity Level 1). At this, the lowest level, there are essentially no sound software engineering management practices in place. Instead, everything is done on an ad hoc basis. If one specific project happens to be staffed by a competent manager and a good software development team, that project may be successful. However, the usual pattern is time and cost overruns caused by a lack of sound management in general, and planning in particular. As a result, most activities are responses to crisis, rather than preplanned tasks. In maturity level 1 organizations, the software process is unpredictable, because it depends totally on the current staff; as the staff changes, so does the process. As a consequence, it is impossible to predict, with any accuracy, the important items such as the time it will take to develop a product or the cost of that product. It is unfortunate that the vast majority of software organizations all over the world are level 1 organizations.

2. Repeatable (Maturity Level 2). At this level, policies for managing a software project and procedures to implement those policies are established. Planning and managing new projects is based on experience with similar projects. An objective in achieving level 2 is to institutionalise effective management processes for software projects, which allow organizations to repeat successful practices developed on earlier projects, although the specific processes implemented by the projects may differ. An effective process can be characterized as practiced, documented, enforced, trained, measured, and amenable to improvement.

Instead of functioning in crisis mode as in level 1, managers identify problems as they arise and take immediate corrective action to prevent them from becoming crises. The key point is that, without measurement it is impossible to detect problems before they get out of hand. Measurements may include

the careful tracking of costs, schedules and functionality. Also, measurements taken during one project can be used to draw up realistic duration and cost schedules for future projects.

Projects in level 2 organizations have installed basic software management controls. Realistic project commitments are based on the results observed on previous projects and on the requirements of the current projects. Software project standards are defined, and the organization ensures that they are faithfully followed. The software project team works with its subcontractors, if any, to establish a strong customer-supplier relationship. The software process capability of level 2 organizations can be summarized as disciplined, because planning and tracking of the software project is stable and earlier successes can be repeated. The project's process is under the effective control of a project management system, following realistic plans based on the performance of previous projects.

3. Defined (Maturity Level 3). At this level, the standard process for developing and maintaining software across the organization is documented, including both software engineering and management processes. This standard process is referred to throughout the CMM as the organization's standard software process. Processes established at level 3 are used, and changed as appropriate, to help the software managers and technical staff to perform more effectively. The organization exploits effective software engineering practices while standardizing its processes. An organization-wide training program is implemented to ensure that the staff and managers have the knowledge and skills required to fulfil their assigned roles.

Projects tailor the organization's standard software process to develop their own defined software process, which accounts for the unique characteristics of the project. This tailored process is referred to, in the CMM, as the project's defined software process. A defined software process contains a coherent, integrated set of well-defined software engineering and management processes. Because the software process is well defined, management has good insight into technical progress of all projects. The software process capability of level 3 organizations can be summarized as standard and consistent, because both software engineering and management activities are stable and repeatable. Within established product lines, cost, schedule, and functionality are under control, and software quality is tracked. This process capability is based on a common, organization-wide understanding of the activities, roles, and responsibilities in a defined software process.

4. Managed (Maturity Level 4). At this level, the organization sets quantitative quality goals for both software products and processes. Productivity and quality are measured for important software process activities across all projects as part of an organizational measurement program. An organization-wide software process database is used to collect and analyze the data available from the projects' defined software processes. Software processes are instrumented with well-defined and consistent measurements at level 4. These measurements establish the quantitative foundation for evaluating the project's software processes and products. Projects achieve control over their products and processes by narrowing the variation in their process performance to fall within acceptable quantitative boundaries.

Meaningful variations in process performance can be distinguished from random variation, particularly within established product lines. The risks involved in moving up the learning curve of a new application domain are known and carefully managed. One measure could be the number of faults detected per 1000 lines of code. A corresponding objective is to reduce this quantity, the number of faults detected, over time. The software process capability of level 4 organizations can be summarized as predictable, because the process is measured and operates within measurable limits. This level of process capability allows an

organization to predict trends of process and product quality within the quantitative bounds of these limits. When these limits are exceeded, action is taken to correct the situation. Software products are of predictably high quality.

5. Optimizing (Maturity Level 5). At this level, the entire organization is focused on continuous process improvement. The organizations have the means to identify weaknesses and strengthen the process proactively, with the goal of preventing the occurrence of defects. Data of the effectiveness of the software process is used to perform cost benefit analysis of new technologies and proposed changes to the organization's software process. Innovations that exploit the best software engineering practices are identified and transferred throughout the organization. Software project teams in level 5 organizations analyze defects to determine their causes. Software processes are evaluated to prevent known types of defects from recurring, and lessons learned are disseminated to other projects.

The software process capability of level 5 organizations can be characterized as continuously improving, because level 5 organizations are continuously striving to improve the range of their process capability, thereby improving the process performance of their projects. Improvement occurs both by incremental advancements in the existing process and by innovations using new technologies and methods.

Table 7.12: The five levels of CMM

Maturity level	Characterization
1. Initial	Ad hoc process
2. Repeatable	Basic project management
3. Defined	Process definition
4. Managed	Process measurement
5. Optimizing	Process control

Experience with the capability maturity model has shown that advancing a complete maturity level usually takes from 18 months to 3 years, but moving from level 1 to level 2 sometimes takes 3 or even 5 years. This is a reflection of how difficult it is to install a methodical approach in an organization that up to now has functioned on a purely ad hoc and reactive basis.

7.4.2 Key Process Areas

Except for level 1, each maturity level is decomposed into several key process areas that indicate the areas an organization should focus on to improve its software process. Key process areas identify the issues that must be addressed to achieve a maturity level. Each key process area identifies a cluster of related activities that, when performed collectively, achieve a set of goals considered important for enhancing process capability. The key process areas and their purposes are listed below, and the name of each key process area is followed by its two-letter abbreviation. By definition, there are no key process areas for level 1.

Table 7.13: Key process areas of the CMM

Level	Key process area	Abbrev.	Purpose
2	Requirements Management	RM	Establish a common relationship between the customer requirements and the developers in order to understand the requirements of the project.
2	Software Project Planning	PP	Establish reasonable plans for performing the software engineering and for managing the software project.
2	Software Project	PT	Establish adequate visibility into actual progress so that management can

	Tracking and Oversight		take effective actions when the software project's performance deviates significantly from the software plans.
2	Software Subcontract Management	SM	Select qualified software subcontractors and manage them effectively.
2	Software Quality Assurance	QA	Provide management with appropriate visibility into the process being used by the software project and of the products being built.
2	Software Configuration Management	CM	Establish and maintain the integrity of the products of the software project throughout the project's software life cycle.
3	Organization Process Focus	PF	Establish the organizational responsibility for software process activities that improve the organization's overall software process capability.
3	Organization Process Definition	PD	Develop and maintain a usable set of software process assets that improve process performance across the projects and provide a basis for cumulative, long-term benefits to the organization.
3	Training Program	TP	Develop the skills and knowledge of individuals so that they can perform their roles effectively and efficiently.
3	Integrated Software Management	IM	Integrate the software engineering and management activities into a coherent, defined software process that is tailored from the organization's standard software process and related process assets.
3	Software Product Engineering	PE	Consistently perform a well-defined engineering process that integrates all the software engineering activities to produce correct, consistent software products effectively and efficiently.
3	Intergroup Coordination	IC	Establish a means for the software engineering group to participate actively with the other engineering groups so the project is better able to satisfy the customer's needs effectively and efficiently.
3	Peer Reviews	PR	Remove defects from the software work products early and efficiently. An important corollary effect is to develop a better understanding of the software work products and of the defects that can be prevented.
4	Quantitative Process Management	QP	Control the process performance of the software project quantitatively.
4	Software Quality Management	QM	Develop a quantitative understanding of the quality of the project's software products and achieve specific quality goals.
5	Defect Prevention	DP	Identify the causes of defects and prevent them from recurring.
5	Technology Change Management	TM	Identify beneficial new technologies, that is, tools, methods, and processes, and transfer them into the organization in an orderly manner.
5	Process Change Management	PC	Continually improve the software processes used in the organization with the intent of improving software quality, increasing productivity, and decreasing the cycle time for product development.

The key process areas at level 2 focus on the software project's concerns related to establishing basic project management controls. The key process areas at level 3 address both project and organizational issues, as the organization establishes an infrastructure that institutionalises effective software engineering and management processes across all projects. The key process areas at level 4 focus on establishing a quantitative understanding of both the software process and the software work products being built. The key process areas at level 5 cover the issues that both the organization and the projects must address to implement continuous and measurable software process improvement.

7.4.3 Common Features

For convenience, common features organize each of the key process areas. The common features are attributes that indicate whether the implementation and institutionalization of a key process area are

effective, repeatable, and lasting. The five common features, followed by their two-letter abbreviations, are listed below.

Commitment to Perform (CO). Describes the actions the organization must take to ensure that the process is established and will endure. It includes practices on policy and leadership.

Ability to Perform (AB). Describes the preconditions that must exist in the project or organization to implement the software process competently. It includes practices on resources, organizational structure, training, and tools.

Activities Performed (AC). Describes the role and procedures necessary to implement a key process area. It includes practices on plans, procedures, work performed, tracking, and corrective action.

Measurement and Analysis (ME). Describes the need to measure the process and analyze the measurements. It includes examples of measurements.

Verifying Implementation (VE). Describes the steps to ensure that the activities are performed in compliance with the process that has been established. It includes practices on management reviews and audits.

The five common features are a checklist that turns a key process area from a label into an auditable set of practices. A project may claim that it performs peer reviews, but the claim becomes credible only when there is a policy that requires them, staff trained to conduct them, a defined procedure, records of the defects found, and an audit that confirms the records are real. The common features are what make the difference between a practice that is performed once by an enthusiastic team and a practice that is institutionalised in the organization.

There is another reason for the growing importance of the CMM. Many software organizations have stipulated that any software development organization that wishes to be a subcontractor must conform to CMM level 3 or higher. Thus, there is pressure for organizations to improve the maturity of their software processes.

7.4.4 Process Capability and Process Performance

The CMM distinguishes between the capability of a process and the performance of a process. Process capability describes the range of expected results that can be achieved by following a process, and it is a property of the organization. Process performance is the actual result achieved by a particular project that follows the process, and it is a property of the project. A level 3 organization has a standard process that gives it a predictable capability, but an individual project may still perform below that capability if it is staffed poorly or if its defined process is badly tailored. The distinction matters because maturity is not a guarantee of success on every project: it is a statement about the range within which results are expected to fall.

The distinction also explains why the levels are defined by institutionalisation rather than by results. An organization that has a good year because of one exceptional manager has not matured; an organization that repeats a good result with different managers has. This is why the level 2 objective is described as institutionalising effective management processes, and why the definition of an effective process includes the words practiced, documented, enforced, trained, measured, and amenable to improvement.

7.4.5 Benefits and Limitations

The benefits claimed for the CMM follow directly from its structure. A mature process makes the cost and schedule of a project predictable, because the organization has historical data and a defined way of using it. It makes quality measurable, because the level 4 organization collects process and product measurements and acts when they fall outside their limits. It makes improvement systematic, because the level 5 organization analyses defects and changes the process rather than only the product. It also makes the organization easier to evaluate from outside, which is why customers began to require a CMM level from their suppliers.

The model has been criticized on several grounds. It is a model of process maturity, not a model of software engineering, and it says nothing about the technical quality of a design or the appropriateness of a technology. Its assessments can become an exercise in producing documentation for the appraiser rather than in changing the way work is done. Its key practices were written with large organizations and long projects in mind, and small organizations often find the documentation burden disproportionate. Finally, the model measures the institutionalisation of practices, not their value: an organization can be mature in the CMM sense and still choose practices that are wrong for its domain.

These criticisms led to the development of the Capability Maturity Model Integration, usually abbreviated CMMI, which integrated the original software CMM with models for systems engineering, acquisition and supplier management. CMMI keeps the idea of maturity levels but renames the key process areas as process areas and offers two representations: a staged representation, which preserves the five levels, and a continuous representation, which allows an organization to improve individual process areas in the order that suits its business. The vocabulary changed, but the underlying claim did not: the capability of an organization's process is a better predictor of its future performance than the heroism of any single project team.

7.5 ISO 9000

As explained in the previous section, the SEI capability maturity model initiative is an attempt to improve software quality by improving the process by which software is developed. The term maturity is essentially a measure of the goodness of the process itself. A different attempt to improve software quality is based on International Standards Organization (ISO) 9000 series standards.

The standard is important, as it is becoming the main focus in which customers can judge the competence of a software developer. It has been adopted for use by over 130 countries. One of the problems with ISO 9000 series standard is that it is not industry-specific. It is expressed in general terms, and can be interpreted by the developers to diverse products such as ball bearings, hair dryers, automobiles, televisions as well as software.

The ISO 9000 series of standards is a set of documents dealing with quality systems that can be used for quality assurance purposes. ISO 9000 series is not just software standard. It is a series of five related standards that are applicable to a wide variety of industrial activities, including design/development, production, installation, and servicing. Within the ISO 9000 series, standard ISO 9001 for quality system is the standard that is most applicable to software development.

Because of the broadness of ISO 9001, ISO has published specific guidelines to assist in applying ISO 9001 to software, namely ISO 9000-3. There is significant room for interpretation in using ISO 9001 in

the software world. ISO 9000-3 is a guide to interpret ISO 9001, yet many-to-many relationships between their clauses raises the suspicion that liberties have been taken in creating this guidance.

7.5.1 Mapping ISO 9001 to the CMM

There are 20 clauses in ISO 9001, which are summarized and compared to the practices in the CMM in this section.

Table 7.14: ISO 9001 clauses and their coverage by the CMM

No.	ISO 9001 requirement	How the CMM addresses it
1	Management responsibility: the quality policy must be defined, documented, understood, implemented and maintained; responsibilities, authorities and verification resources must be defined; a designated manager ensures the quality programme is implemented.	Addressed primarily in Software Quality Assurance; project planning and tracking also identify responsibility for performing project roles.
2	Quality system: a documented quality system, including procedures and instructions, must be established; ISO 9000-3 describes it as an integrated process throughout the life cycle.	Addressed in Software Quality Assurance; the procedures are distributed throughout the key process areas in the Activities Performed practices.
3	Contract review: contracts must be reviewed to determine whether requirements are adequately defined, agreed with the bid, and can be implemented.	Addressed in Requirements Management, which ensures that system requirements allocated to software are documented and reviewed.
4	Design control: procedures to control and verify the design must be established, including design inputs and outputs, verification, and control of design changes.	Life cycle activities are described in Software Product Engineering, plans in Software Project Planning and Tracking, and control of work products in Software Configuration Management.
5	Document control: the distribution and modification of documents must be controlled.	Addressed in Software Configuration Management; specific documents under control are distributed through the Activities Performed practices.
6	Purchasing: purchased products must conform to specified requirements, including assessment of subcontractors and verification of purchased products.	Addressed in Software Subcontract Management.
7	Purchaser-supplied product: any purchaser-supplied material must be verified and maintained.	Partly addressed in Integrated Software Management, where off-the-shelf and reusable software are considered during planning; this is one of the areas where the CMM is weak.
8	Product identification and traceability: the product must be identified and traceable during all stages of production, delivery and installation.	Covered primarily in Software Configuration Management.
9	Process control: production processes must be defined and planned, carried out under controlled conditions, according to documented instructions; special processes must be continuously monitored.	The procedures defining the software production process are distributed through the Activities Performed practices; standards are specified in the software development plan.
10	Inspection and testing: incoming materials must be inspected or certified, in-process inspection and testing must be performed, and final inspection and testing must precede release.	Testing is described in Software Product Engineering; in-process inspections are addressed in Peer Reviews.
11	Inspection, measuring and test equipment: equipment used to demonstrate conformance must be controlled, calibrated and maintained.	Generally addressed in the testing practices of Software Product Engineering.
12	Inspection and test status: the status of inspections and tests must be maintained for items as they progress through processing steps.	Addressed by testing practices in Software Product Engineering and by problem reporting and configuration status in Software Configuration Management.
13	Control of nonconforming product: nonconforming product	Design, implementation, testing and validation

	must be controlled to prevent inadvertent use or installation.	are addressed in Software Product Engineering; configuration status in Software Configuration Management; installation is not addressed in the CMM.
14	Corrective action: the causes of nonconforming product must be identified and eliminated, and procedures changed as a result.	Problem reporting is described in Software Configuration Management; the analysis of causes and the updating of the process are weaker in the CMM.
15	Handling, storage, packaging and delivery: procedures must be established and maintained; replication, delivery and installation are not covered in the CMM.	Not covered by the CMM.
16	Quality records: quality records must be collected, maintained and disposed of.	Records are kept through the Activities Performed practices of the key process areas.
17	Internal quality audits: audits must be planned and performed, results communicated to management, and deficiencies corrected.	The auditing process is described in Software Quality Assurance; specific audits are called out in the Verifying Implementation common feature.
18	Training: training needs must be identified, training provided, and records maintained.	Training needs are identified in the training and orientation practices of the Ability to Perform common feature.
19	Servicing: servicing activities must be performed as specified.	Maintenance is embedded throughout the practices of the CMM rather than being a separate process; the practices do not directly address the unique aspects of the maintenance environment.
20	Statistical techniques: appropriate statistical techniques must be identified and used to verify the acceptability of process capability and product characteristics.	Product measurement is incorporated into the Activities Performed practices; process measurement is described in the Measurement and Analysis common feature.

7.5.2 Contrasting ISO 9001 and the CMM

Clearly there is a strong correlation between ISO 9001 and the CMM, although some issues in ISO 9001 are not covered in the CMM, and some issues in the CMM are not addressed in ISO 9001.

The biggest difference, however, between these two documents is the emphasis of the CMM on continuous process improvement. ISO 9001 addresses the minimum criteria for an acceptable quality system. It should also be noted that the CMM focuses strictly on software, while ISO 9001 has a much broader scope: hardware, software, processed materials, and services.

The biggest similarity is that for both the CMM and ISO 9001, the bottom line is say what you do and do what you say. The fundamental premise of ISO 9001 is that every important process should be documented and every deliverable should have its quality checked through a quality control activity. ISO 9001 requires documentation that contains instructions or guidance on what should be done or how it should be done. The CMM shares this emphasis on processes that are documented and practised as documented. Phrases such as conducted according to a documented procedure and following a written organizational policy characterize the key process areas in the CMM.

The CMM also emphasizes the need to record information for later use in the process and for improvement of the process. This is equivalent to the quality records of ISO 9001 that document whether or not the required quality is achieved and whether or not the quality system operates effectively.

7.5.3 Conclusion

Although there are specific issues that are not adequately addressed in the CMM, in general the concerns of ISO 9001 are encompassed by the CMM. The converse is less true. ISO 9001 describes the minimum criteria for an adequate quality management system rather than process improvement, although future revisions of ISO 9001 may address this concern. The differences are sufficient to make a route mapping impractical, but the similarities provide a high degree of overlap.

Should software process improvement be based on the CMM, with perhaps some extension for ISO 9001 specific concerns, or should the improvement effort focus on certification concerns? A market may require ISO 9001 certification, and a level 1 organization would certainly profit from addressing the concerns of ISO 9001. It is also true that addressing the concerns of the CMM would help organizations prepare for an ISO 9001 audit. Although either document could be used to structure a process improvement program, the more detailed guidance and greater breadth provided to software organizations by the CMM suggest that it is the better choice, perhaps a biased answer.

In any case, building competitive advantage should be focused on improvement, not on achieving a score, whether the score is a maturity level or a certificate. We would advocate addressing the larger context encompassed by the CMM, but even then there is a need to address the still larger business context, as exemplified by Total Quality Management.

7.5.4 Obtaining ISO 9000 Certification

An organization intending to obtain ISO 9000 certification applies to an ISO 9000 registrar for registration. The registration process consists of six stages, and it is worth following them in order because each stage produces evidence that the next stage examines.

1. **Application.** Once an organization decides to go for ISO 9000 certification, it applies to a registrar for registration.
2. **Pre-assessment.** During this stage the registrar makes a rough assessment of the organization, in order to establish whether a full assessment is likely to succeed.
3. **Document review and adequacy of audit.** During this stage the registrar reviews the documents submitted by the organization and makes suggestions for possible improvements.
4. **Compliance audit.** During this stage the registrar checks whether the suggestions made during the review have been complied with by the organization.
5. **Registration.** The registrar awards the ISO 9000 certificate after the successful completion of all the previous phases.
6. **Continued surveillance.** The registrar continues to monitor the organization, although only periodically.

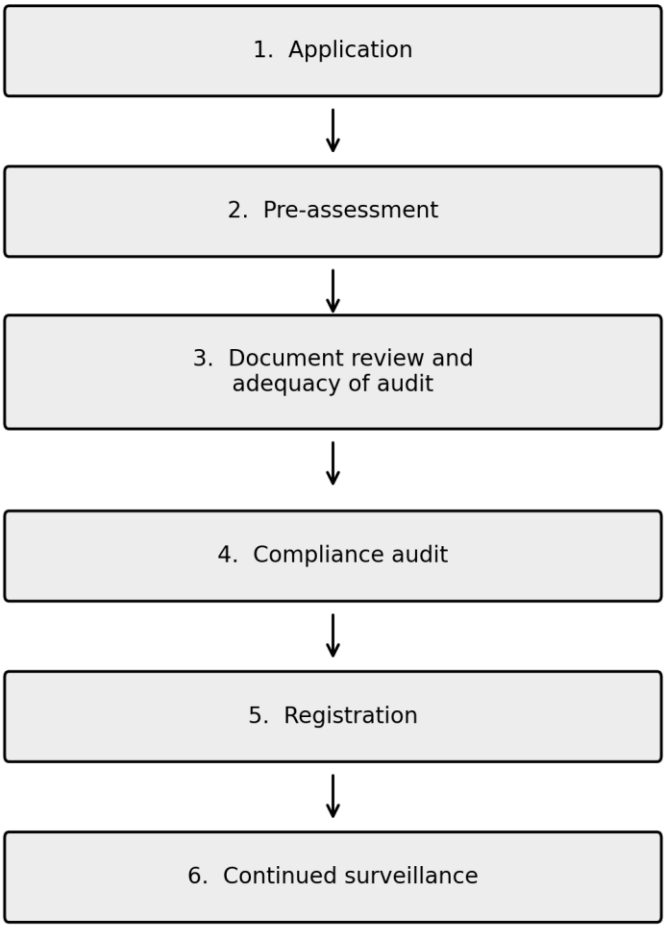


Figure 7.24. The six stages of ISO 9000 registration.

The sequence explains why the certificate is not a one-off examination. The document review and the compliance audit are separated by a period in which the organization is expected to act on the registrar's findings, and the surveillance stage means that the certificate can be withdrawn if the system later stops being followed. The certificate is therefore a statement about a system that is maintained, not about a product that was once inspected.

7.5.5 Benefits and Limitations of ISO 9000 Certification

The benefits claimed for ISO 9000 certification follow from the discipline that the standard imposes.

Continuous improvement. Certification forces an organization to focus on how it does business. Each procedure and work instruction must be documented, and the documentation becomes the springboard for continuous improvement.

Elimination of variation. Documented processes are the basis for repetition and help eliminate variation within the process. As variation is eliminated, efficiency improves; as efficiency improves, the cost of quality is reduced.

Higher real and perceived quality. With the development of solid corrective and preventive measures, permanent, company-wide solutions to quality problems are found, and this results in higher quality.

Improved morale and participation. Employee morale increases as people are asked to take control of their processes and to document their work, and many problems are reduced by increased employee participation, involvement, awareness and systematic training.

Improved customer satisfaction. Customer satisfaction, and more importantly customer loyalty, grows as a company transforms from a reactive organization to a proactive, preventive organization.

Better products and services. Better products and services result from continuous improvement processes.

Greater quality assurance. Certification fosters the understanding that quality is not limited to a quality department but is everyone's responsibility.

Improved profit levels. The reduction of variation and of rework has a direct effect on cost, and therefore on profit.

The limitations are equally important, and they are often omitted from the promotional literature.

First, ISO 9000 does not provide any guidelines for defining an appropriate process. It states what a quality system must contain, but it does not say which development process a software organization should use, and it therefore cannot by itself make a process better.

Second, the certification process is not foolproof, and no international accrediting agency exists. The quality of an audit depends on the registrar, and a certificate issued by one body is not necessarily equivalent to a certificate issued by another.

Third, ISO 9000 does not automatically lead to continuous process improvement. It does not automatically lead to total quality management. An organization can satisfy the standard and then stop improving, because the standard describes a minimum, not a trajectory.

These limitations explain the uses to which the standard is actually put. ISO certification is used as a reference model for a contract between external independent parties, and it is now used as the standard and basis of evaluation for international bidding. It must be remembered that ISO certification does not guarantee a high-quality product: it focuses mainly on the processes. What it does guarantee is that the organization has a documented system and can show evidence that the system is followed.

7.5.6 Salient Features of ISO 9001 Requirements

The requirements of ISO 9001 can be summarized in four salient features, each of which corresponds to a family of clauses examined earlier.

- All documents concerned with the development of a software product should be properly managed, authorized, and controlled.
- Proper plans should be prepared, and progress against these plans should be monitored.

- Important documents should be independently checked and reviewed for effectiveness and correctness.
- The product should be tested against its specifications.

The four features are deliberately modest. They do not require a particular life cycle, a particular notation, or a particular tool; they require that the work be planned, that the plans be followed, that the documents be controlled, that the important ones be reviewed independently, and that the product be tested against what it was supposed to do. An organization that does these four things has a quality system in the sense of ISO 9001, whatever process model it uses to develop its software. An organization that does them only when the auditor is present has a certificate, but not a quality system.

7.5.7 The ISO 9000 Family and Its Mission

The mission of the ISO is to promote the development of standardization and related activities to facilitate the international exchange of goods and services, and to develop cooperation in the spheres of intellectual, scientific, technological and economic activity. The ISO published its ISO 9000 standard in 1988. ISO 9000 consists of three standards for external quality assurance: ISO 9001, ISO 9002 and ISO 9003.

ISO 9001. ISO 9001 is an international quality-management system. It is mainly related to the software industry: it lays down the standards for designing, developing, servicing and producing a standard quality of goods, and it is applicable to most software-development organizations.

ISO 9002. ISO 9002 is basically related to manufacturing only and is silent on design issues. Examples of this category of industries include steel and car manufacturing, which buy the product and plant designs from external sources and are involved only in manufacturing those products. ISO 9002 is therefore not applicable to software-development organizations.

ISO 9003. ISO 9003 applies to the service industry. Organizations that are involved only in the installation of products, services and testing of products are eligible for ISO 9003 certification.

Table 7.17: The three ISO 9000 standards for external quality assurance

Standard	Scope	Applicability to software
ISO 9001	Design, development, servicing and production of goods	Applicable to most software-development organizations
ISO 9002	Manufacturing only; silent on design	Not applicable, because software development always includes design
ISO 9003	Installation, servicing and testing only	Applicable only to organizations that install or test software produced by others

The family structure explains a confusion that is common in the software industry. When an organization says that it is ISO 9000 certified, it usually means that it holds an ISO 9001 certificate, because ISO 9001 is the standard that covers design. A firm that only installs or tests products may hold an ISO 9003 certificate, and a firm that only manufactures may hold an ISO 9002 certificate, but neither of those certificates says anything about the design of software.

The date is worth noting. The standard appeared in 1988, before the CMM had become the dominant vocabulary of software process improvement, and it was written for manufacturing. Its application to software required a separate guide, ISO 9000-3, and that guide is the source of the many-to-many

relationships between the clauses that were noted earlier. The later revisions of the standard moved it closer to the process view, but the 1988 structure is the one that the chapter describes.

7.5.8 ISO 9000 Certification and the SEI-CMM Compared

The comparison between the two frameworks is often reduced to a slogan, but the differences are specific and they matter to an organization that has to choose where to spend its improvement effort.

The emphasis of the SEI-CMM is on continuous process improvement, whereas ISO 9000 addresses the minimum criteria for an acceptable quality system. The CMM is a five-level framework for measuring software-engineering practices as they relate to a process; ISO 9000 defines a minimum level of generic attributes for a quality-management programme. One describes a ladder, the other describes a floor.

The second difference concerns who awards the result. ISO 9000 is awarded by an international standards body, and therefore ISO 9000 certification can be quoted by an organization in official documents, in communication with external parties and in tender quotations. SEI-CMM assessment, by contrast, is purely for internal use. An organization can be assessed at level 3 and keep the result private; it cannot advertise an ISO certificate that it does not hold.

The third difference concerns origin. The ISO 9000 standards were basically designed to audit manufacturing and service organizations, whereas the CMM was developed specifically for the software industry and therefore addresses several issues specific to software, such as the management of requirements, the control of the software configuration and the conduct of peer reviews. This is why the mapping of Section 7.5.1 is not one-to-one: a clause written for a factory does not always have a natural counterpart in a software process.

Table 7.18: ISO 9000 certification and the SEI-CMM compared

Aspect	ISO 9000	SEI-CMM
Goal	Minimum criteria for an acceptable quality system	Continuous process improvement through five maturity levels
Structure	A set of clauses describing what the quality system must contain	A hierarchy of maturity levels, key process areas and common features
Scope	Generic: hardware, software, processed materials and services	Specific to software engineering
Award	Certificate issued by an accredited registrar	Assessment, normally for internal use
Use	Contractual reference, international bidding, public quotation	Internal planning and improvement of the software process
Limitation	Does not prescribe a process and does not by itself lead to continuous improvement	Does not cover delivery, installation and servicing explicitly; assessment is not a public certificate

The table explains why the two frameworks are complementary rather than competing. An organization that needs to satisfy a customer's contractual requirement will seek ISO 9001 certification; an organization that wants to improve the way it develops software will use the CMM or CMMI to decide what to improve next. The first answers the question what must we be able to show, and the second answers the question what should we do better. A mature organization eventually answers both, but it does not confuse the two answers.

7.5.9 Why the Software Industry Seeks Certification

There are several reasons why the software industry must obtain ISO certification. Some of the important reasons are the following.

It is a sign of customer confidence. The certification has become a standard for international bidding, and a supplier without it may be excluded before the technical evaluation begins.

It is a motivating factor for business organizations. The preparation for an audit gives a management team a reason and a deadline to put its procedures in order, and a deadline is often what converts an intention into a change.

It makes processes more focused, efficient and cost-effective. The act of documenting a process exposes the steps that add no value, and the documented process is easier to repeat than the one that exists only in the memory of the staff.

It helps in designing high-quality repeatable software products. Repetition requires a defined process, and a defined process is the precondition for improving quality release after release.

It highlights weaknesses and suggests corrective measures for improvement. The document review and the compliance audit are, in effect, an external review of the quality system, and an external review sees things that an internal review has learned to ignore.

It facilitates the development of optimal processes and total quality measurement. Certification provides the baseline on which a later programme of measurement and improvement can be built.

It emphasizes the need for proper documentation. Documentation is the visible evidence of the process, and the certification process makes that evidence a requirement rather than a preference.

The list is worth reading critically. Most of the benefits are benefits of the preparation, not of the certificate. The certificate is the receipt; the discipline that produced it is the asset. An organization that treats the certificate as the goal will stop improving as soon as the auditor leaves, and it will then hold a document that describes a system it no longer follows. An organization that treats the preparation as the beginning will use the same discipline to measure its process, to find its weaknesses and to improve them, which is precisely what the capability maturity model asks it to do.